



POLITECHNIKA POZNAŃSKA

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI
Instytut Sieci Teleinformatycznych

Praca dyplomowa inżynierska

ANALIZA SKUTECZNOŚCI WYKRYWANIA ATAKÓW ZERO-DAY W DANYCH SIECIOWYCH Z WYKORZYSTANIEM UCZENIA MASZYNOWEGO NIENADZOROWANEGO

Autor: Michał Kowalski, nr-indeksu: 155984

Promotor
dr hab. inż. Sławomir Hanczewski

POZNAŃ 2026

Spis treści

1	Wstęp	2
2	Wprowadzenie	3
2.1	Cyberbezpieczeństwo i ataki zero-day	3
2.2	Rola uczenia maszynowego w wykrywaniu zagrożeń	4
3	Uczenie nienadzorowane i wykrywanie anomalii	6
3.1	Podstawy uczenia maszynowego	6
3.2	Algorytmy: Isolation Forest, Local Outlier Factor, Autoencoder	7
4	Implementacja Isolation Forest	11
5	Implementacja Local Outlier Factor	15
6	Implementacja Autoencodera	19
6.1	Użyte zasoby	19
6.2	Implementacja	19
6.3	Użyta architektura	20
6.4	Proces uczenia	21
7	Dane i przygotowanie środowiska	23
7.1	Zbiór danych CICIDS2017	23
7.2	Ataki zawarte w CICIDS2017	23
7.3	Charakterystyka zbioru danych CICIDS2017	27
7.4	Przygotowanie i przetwarzanie danych	27
7.5	Proces przetwarzania danych	30
7.6	Środowisko badawcze	31
8	Badania i analiza wyników	33
8.1	Metody oceny wydajności	33
8.2	Wyniki modelu Isolation Forest	34
8.3	Wyniki modelu Local Outlier Factor	35
8.4	Wyniki modelu Autoencoder	37
9	Wnioski	39
9.1	Podsumowanie badań	39
9.2	Kierunki dalszych prac	40
	Spis rysunków	44

Spis tabel	45
Literatura	46

Streszczenie

W pracy został poruszony temat wykorzystania metod uczenia maszynowego nienadzorowanego w wykrywaniu ataków zero-day poprzez sygnatury anomalii w ruchu sieciowym. Omówiono konkretną implementację środowiska testowego mającego na celu sprawdzenie skuteczności trzech różnych algorytmów wykrywania anomalii poprzez uczenie nienadzorowane: Isolation Forest, Local Outlier Factor i Autoencoder. Następnie przedstawiono wyniki implementacji i określono jakość detekcji.

Abstract

The work addresses the topic of utilizing unsupervised machine learning methods in detecting zero-day attacks through anomaly signatures in network traffic. A specific implementation of a testing environment was discussed, aimed at verifying the effectiveness of three different anomaly detection algorithms through unsupervised learning: Isolation Forest, Local Outlier Factor, and Autoencoder. Subsequently, the implementation results were presented and the detection quality was determined.

Rozdział 1

Wstęp

Celem niniejszej pracy jest szczegółowe przybliżenie możliwości, jakie oferuje uczenie maszynowe, oraz omówienie jego praktycznego wykorzystania w dziedzinie cyberbezpieczeństwa. W pracy termin atak zero-day rozumiany jest jako atak, którego charakterystyka nie jest znana na etapie uczenia modelu, a nie jako konkretna, zidentyfikowana podatność opisana w bazach CVE. W pierwszej sekcji pracy szczegółowo omówiono technologie, które zaimplementowano w specjalnie przygotowanym środowisku badawczym. Przybliżony zostanie aktualny kontekst światowy oraz narastającą potrzebą stworzenia innowacyjnych rozwiązań, które skutecznie chroniłyby systemy informatyczne przed coraz bardziej zaawansowanymi cyberatakami.

W dalszej części pracy przedstawiono szczegółową charakterystykę technologii wykorzystywanej w trakcie przeprowadzonych badań. Dokładnie wyjaśnione zostanie zastosowanie poszczególnych algorytmów uczenia maszynowego oraz fundamentalny zamysł ich działania, co pozwoli na pełne zrozumienie mechanizmów leżących u podstaw badań.

W kolejnej sekcji przedstawiony zostanie wyczerpujący opis przeprowadzonych badań eksperymentalnych, rozpoczynając od szczegółowej i wieloaspektowej analizy bazy danych ruchu sieciowego. Zaprezentowano również uzasadnienie procesu wyodrębnienia najważniejszych cech oraz kluczowych właściwości przepływów pakietów, co w konsekwencji umożliwiło najbardziej efektywne oraz optymalne wykorzystanie dostępnych zasobów obliczeniowych. Określone zostaną szczegółowe parametry zastosowanych modeli oraz przedstawione zostanie merytoryczne uzasadnienie podjętych decyzji dotyczących ich wyboru. W końcowym etapie pracy przybliżono wyniki uzyskane z przeprowadzonych badań. Porównane zostaną zarówno poszczególne modele, jak i różnorodne sposoby ich wzajemnego połączenia, które w efekcie pozwalają na uzyskanie najlepszego możliwego wyniku w kontekście analizowanego problemu.

Rozdział 2

Wprowadzenie

2.1 Cyberbezpieczeństwo i ataki zero-day

Cyberbezpieczeństwo (ang. cybersecurity) to złożona dziedzina badająca ochronę systemów i sieci komputerowych. Na przestrzeni lat termin ten ewoluował, by lepiej oddawać zakres i znaczenie tej nauki. Współczesna nazwa powstała z połączenia słowa „bezpieczeństwo” z przedrostkiem „cyber”, wywodzącym się z cybernetyki, nauki o sterowaniu i komunikacji w maszynach oraz organizmach żywych. Wraz z rozwojem technologii cyfrowych przedrostek „cyber” zaczął określać zjawiska związane z Internetem i komputerami.

Dla pełnego zrozumienia pojęcia warto wyjaśnić termin „cyberprzestrzeń”, ściśle powiązany z tematyką badań.

„Termin „cyberprzestrzeń” (ang. cyberspace) został użyty po raz pierwszy w 1984 roku przez W. Gibsona w powieści *Burning Chrome*. Wygenerowany przez komputer świat wirtualnej rzeczywistości stał się szczególną matrycą (matrix), inaczej przestrzenią komunikowania tworzoną przez system powiązań internetowych (Blair, D.C. 2011, s. 2). Cyberprzestrzeń w dzisiejszym rozumieniu to przestrzeń otwartego komunikowania się za pośrednictwem połączonych komputerów i pamięci informatycznych pracujących na całym świecie jako synonim Internetu. Według Pierre’a Levy’ego cyberprzestrzeń ma charakter „plastyczny, płynny, obliczalny z dużą dokładnością i przetwarzalny w czasie rzeczywistym, hipertekstualny, interaktywny i wreszcie wirtualny. To nowe środowisko umożliwia współdziałanie poprzez łączenie wszystkich narzędzi tworzenia informacji, rejestrowania, komunikacji i symulacji, przez co stało się głównym kanałem informacyjnym i głównym nośnikiem pamięciowym ludzkości” [1].

Cyberbezpieczeństwo odnosi się do ochrony informacji, systemów i sieci funkcjonujących w cyberprzestrzeni. Obejmuje zapewnienie poufności i dostępności danych oraz odporność infrastruktury cyfrowej na ataki i zakłócenia.

Wraz z rozwojem technologii informatycznych pojawiła się nowa klasa zagrożeń. W cyberprzestrzeni funkcjonuje wiele rodzajów ataków, w tym phishing, ransomware, ataki DDoS, przejęcia kont użytkowników, czy eksfiltracja danych. Wspólnym celem tych działań jest uzyskanie nieautoryzowanego dostępu do informacji lub zakłócenie pracy systemów. Wśród nich szczególne miejsce zajmują ataki wcześniej niespotykane lub niezarejestrowane, nazywane atakami zero-day. Wykorzystują one nieznanne wektory ataku na użytkowników bądź systemy sieciowe. Ich wykrywanie i neutralizacja stanowi jedno z największych wyzwań współczesnego cyberbezpieczeństwa.

Przykładem ataku zero-day jest „2021 Microsoft Exchange Server data breach”. W 2021 roku grupa hakerska wykorzystwała cztery luki zero-day w serwerach On-premises Exchange należących do firmy Microsoft. Owe luki w zabezpieczeniach zostały dokładnie opisane w bazie danych „Con-

tinuous vulnerability assessment” (CVA) pod nazwami *CVE-2021-26855* [2], *CVE-2021-26857* [3], *CVE-2021-26858* [2] i *CVE-2021-27065* [4], [5].

W praktyce oznaczało to, że atakujący mogli uzyskać dostęp do serwerów i instalować na nich złośliwe oprogramowanie, zanim Microsoft zdążył wydać poprawki. Każda z luk zostawiała w sieci swój ślad. Na przykład *CVE-2021-26855* pozwalała wysyłać nietypowe żądania do serwera, co w logach HTTP wyglądało inaczej niż normalny ruch pocztowy. *CVE-2021-26857* umożliwiała uruchomienie złośliwego kodu, co powodowało nagłe połączenia z serwerami kontrolowanymi przez hakerów. *CVE-2021-26858* i *CVE-2021-27065* pozwalały na zapis plików na serwerze, np. webshelli, które generowały dziwne zapytania HTTP do rzadko używanych ścieżek [5], [2], [3], [4].

Z punktu widzenia monitorowania sieci oznacza to, że nawet jeśli luka była nieznana, jej wykorzystanie mogło być wykryte poprzez obserwowanie nietypowego ruchu. System, który potrafi rozpoznać odchylenia od normalnego zachowania, mógłby w porę ostrzec administratora o podejrzanym aktywności. Innymi słowy, atak wykorzystywał nieznane błędy w oprogramowaniu, ale nie ukrywał swojej obecności w sieci. Właśnie chęć stworzenia systemu ostrzegania o dziejącym się ataku była głównym celem tej pracy.

2.2 Rola uczenia maszynowego w wykrywaniu zagrożeń

ML (ang. Machine Learning) posiada ogromny potencjał. Nie tylko w imitowaniu elementów ludzkiego rozumowania, lecz także w wykrywaniu wzorców niedostrzegalnych dla obserwatora. Jako część dziedziny sztucznej inteligencji (ang. Artificial Intelligence) ML ma za zadanie odtworzyć zdolność ludzkiego uczenia się. Jednak replikacja wyniku, zdolności uczenia się z dostępnej bazy informacji, nie oznacza replikacji rzeczywistej metody używanej w ludzkim podejściu. Mimo że definicja ma na celu porównanie konceptu z ”ludzkim” uczeniem się, uczenie maszynowe kopiuje efekt uczenia, zamiast procesu. Określenie, czy jest to zaleta, czy wada, nie jest w zakresie merytorycznym tego, co zbadano w tej pracy. Skupiono się na wykorzystaniu tego efektu.

Tradycyjne metody wykrywania ataków sieciowych bez ML opierają się na sygnaturach i filtracji ruchu. Celem jest redukcja liczby pakietów poddawanych dalszej analizie. Ruch w sieciach lokalnych o małej lub średniej skali łatwo osiąga miliony pakietów na dobę. Nawet przy zastosowaniu zaawansowanych filtrów ilość danych do iteracyjnego przejrzenia pozostaje duża. Administratorzy używają istniejących sygnatur ataków do konstruowania reguł detekcji. Program monitoruje ruch i porównuje bieżące wzorce z zapisanymi sygnaturami. Z czasem takie reguły były zastępowane lub integrowane z modelami ML, które miały lepiej dopasowywać sygnatury ruchu do potencjalnych zagrożeń. Sposób, w jaki model uczy się sygnatur, zależy od projektu systemu. Uogólniając, można wyróżnić podstawowy wektor podejścia.

Najczęściej model otrzymuje zbiór danych składający się ze strumieni (ang. flows) i pakietów pochodzących z zarejestrowanych ataków. Po treningu oczekuje się, że ML będzie mierzył podobieństwo bieżącego ruchu do przykładów z etapu treningowego. Model zwraca wskaźnik podobieństwa lub prawdopodobieństwo. Architekt ustala próg decyzyjny. Jeśli wartość przekroczy ten próg, generowany jest alert [6]. Takie podejście ma kilka zalet. Najważniejszą z nich jest stosunkowo niska liczba fałszywych alarmów (ang. false positives), czyli przypadków, w których system błędnie interpretuje normalne działanie sieci jako próbę ataku. Dzieje się tak dlatego, że system reaguje wyłącznie wtedy, gdy zaobserwowane zachowanie sieci odpowiada uprzednio zarejestrowanej sygnaturze, czyli charakterystycznemu wzorcowi danych, który został wcześniej powiązany z określonym typem ataku. Takie działanie ma jednak istotne ograniczenie. Jeśli dany atak nigdy wcześniej nie został zarejestrowany, system go nie wykryje. Oznacza to, że nowe techniki lub wektory ataku (czyli sposoby, w jakie atakujący może uzyskać dostęp do systemu) nieobecne w zbiorze

treningowym pozostaną niezauważone. Tego rodzaju przypadki określa się mianem podatności dnia zerowego (ang. zero-day vulnerability), ponieważ luka jest nieznana producentowi oprogramowania i nie istnieje jeszcze żadne oficjalne zabezpieczenie.

Znając już podstawową słabość systemu opartego na sygnaturach, można zatem zmienić punkt widzenia. Zamiast uczyć model na przykładach złośliwego ruchu, można trenować go na ruchu normalnym. W takim podejściu nie dokumentuje się każdego możliwego wariantu ataku. AI (ang. Artificial Intelligence) skupia się na modelowaniu „normy” i wykrywaniu odchyłeń od niej. Atak jest anomalią względem normalnego profilu ruchu. Jeśli ruch znacząco odbiega od modelowanej normy, model zgłosi to jako wysoki współczynnik odchylenia lub duże prawdopodobieństwo anomalii. To podejście eliminuje konieczność posiadania kompletnej bazy sygnatur złośliwych zachowań. Jego skuteczność zależy od jakości modelu normalności, stabilności ruchu i sposobu radzenia sobie z dryfem danych. W dalszej części pracy zostanie rozwinięta skuteczność tego podejścia, oraz zaproponowana ścieżka implementacji.

Rozdział 3

Uczenie nienadzorowane i wykrywanie anomalii

3.1 Podstawy uczenia maszynowego

Aby jak najlepiej przedstawić rolę uczenia nienadzorowanego w wykrywaniu zagrożeń, należy najpierw nakreślić koncepcję uczenia maszynowego. Uczenie maszynowe to dziedzina sztucznej inteligencji, która polega na tworzeniu programów zdolnych do pozyskiwania doświadczenia z istniejących danych i uczenia się wzorców w celu przewidywania wyników lub podejmowania decyzji. Początki sięgają badań nad automatycznym uczeniem maszynowym oraz rozwojem sztucznej inteligencji.

Koncept uczenia maszynowego dzieli się na cztery typy. Każdy typ różni się zastosowaniem, jak i głównym konceptem działania [7]:

- **Uczenie nadzorowane** – algorytm korzysta z danych wejściowych wraz z odpowiadającymi im etykietami. Model uczy się przewidywać wyniki dla nowych danych na podstawie wzorców z danych treningowych. W taki sposób działa przykładowo algorytm uczony na podpisanych zdjęciach kotów i psów, aby później samodzielnie rozróżniać zwierzęta.
- **Uczenie nienadzorowane** – algorytm analizuje nieoznakowane dane i samodzielnie identyfikuje wzorce oraz grupy podobnych obserwacji. Stosuje się je m.in. do wykrywania anomalii w zbiorach danych.
- **Uczenie częściowo nadzorowane** – łączy dane oznakowane i nieoznakowane, umożliwiając algorytmowi wykorzystanie ograniczonego zbioru oznakowanych danych do wstępnego uczenia, a następnie dalsze grupowanie danych nienadzorowanych.
- **Uczenie ze wzmocnieniem** – algorytm uczy się poprzez interakcję ze środowiskiem, otrzymując nagrody lub kary w zależności od podjętych działań. Przykładem jest nauka gry w szachy, gdzie system uczy się strategii poprzez wielokrotne próby i analizę skutków ruchów.

Potencjalne zastosowania uczenia maszynowego w cyberbezpieczeństwie mają ogromny potencjał i są szeroko stosowane w różnych jego aspektach. Uczenie maszynowe odgrywa kluczową rolę w identyfikowaniu złośliwych plików, odnośników oraz wiadomości e-mail poprzez analizę zawartości i szukanie wzorców zarejestrowanych jako typowy wektor ataku na użytkownika. W przypadku uczenia nadzorowanego model jest trenowany na oznakowanych zbiorach danych, co pozwala mu rozpoznawać określone typy zagrożeń. Przykładowo, taki model może wykrywać wiadomości phishingowe lub znane odmiany złośliwego oprogramowania. Systemy oparte na uczeniu maszynowym

mogą reagować w czasie rzeczywistym na incydenty, blokując podejrzane adresy IP, izolując zainfekowane urządzenia lub tymczasowo blokując konta użytkowników, które wykazują oznaki kompromitacji. Algorytmy uczenia maszynowego są również w stanie analizować ogromne ilości danych sieciowych w poszukiwaniu nieregularności, takich jak nieoczekiwane transfery danych czy nietypowe próby logowania, które mogą wskazywać na wczesną fazę cyberataku. Automatyczna analiza tego rodzaju danych pozwala znacząco przyspieszyć proces wykrywania zagrożeń oraz skrócić czas reakcji na potencjalne incydenty.

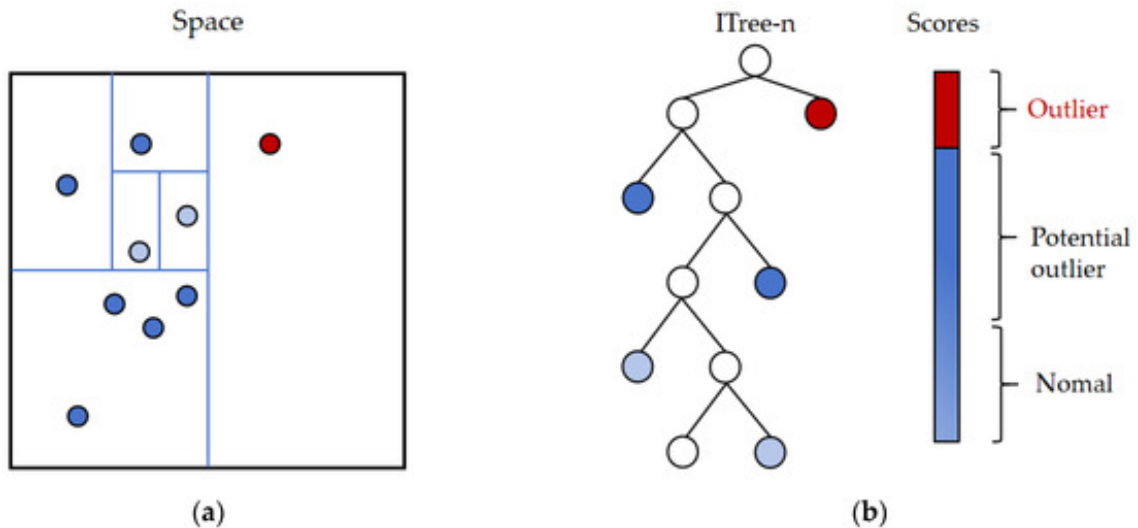
3.2 Algorytmy: Isolation Forest, Local Outlier Factor, Autoencoder

Chcąc skutecznie zbadać możliwości uczenia nienadzorowanego, nie ograniczono się do wykorzystania jednego algorytmu. Aby poprawnie zbadać potencjał, w badaniach skupiono się na trzech różnych algorytmach uczenia nienadzorowanego, które zostały zaimplementowane w celu wykrywania anomalii w ruchu sieciowym.

- **Isolation Forest:**

Isolation Forest (IF) ten bezpośrednio wykorzystuje unikatowe właściwości anomalii aby je wykrywać. Zasada działania różni się od typowego podejścia określenia normy a następnie szukania punktów które od niej odbiegają. W Isolation Forest algorytm zakłada że anomalie są stosunkowo rzadkie i znacząco wyróżniają się od reszty danych. Aby je wykryć wystarczy odpowiednio podzielić i pogrupować przestrzeń danych w taki sposób żeby anomalie same zostały odgraniczone od reszty.

Algorytm wykorzystuje strukturę drzewa binarnego do realizacji procesu izolacji. Działanie polega na rekurencyjnym, losowym wybieraniu cechy oraz wartości podziału, aż do momentu, gdy pojedyncza obserwacja zostanie odseparowana od reszty zbioru(3.1). Kluczową myślą stojącą za algorytmem jest to, że obserwacje, które są anomalią, będą wymagały znacznie mniejszej liczby podziałów. Wizualnie można to przedstawić jako przestrzeń danych, w której normalne obserwacje tworzą zwarte skupiska, podczas gdy anomalie znajdują się na peryferiach. Aby odizolować taką pojedynczą, odległą obserwację, wystarczy kilka cięć, podczas gdy wydzielenie pojedynczej obserwacji z gęstego skupiska wymagać będzie wielu kolejnych podziałów(rys 3.1.a) [8].



RYSUNEK 3.1: Ilustracja działania algorytmu. Na czerwono zaznaczone zostały anomalie a na niebiesko normalne dane. a - cięcia, b- struktura drzewa [8]

Proces budowy pojedynczego drzewa izolacji jest następujący. Dla danego podzbioru danych, algorytm losuje cechę, a następnie wybiera wartość podziału z zakresu wartości tej cechy w aktualnym podzbiorze. Na tej podstawie dzieli obserwacje na dwie grupy. Procedura jest powtarzana rekurencyjnie dla każdej z powstałych grup, aż do momentu, gdy wszystkie obserwacje zostaną odizolowane, czyli każda będzie znajdować się w osobnym liście drzewa, lub gdy osiągnięta zostanie maksymalna, zdefiniowana głębokość drzewa. Pojedyncze drzewo izolacji, ze względu na losowy charakter swojego budowania, nie zapewnia stabilnego ani wiarygodnego wyniku. Dlatego algorytm wykorzystuje podejście zespołowe, konstruując wiele takich drzew, tworząc tym samym las izolacji. Wynikiem działania algorytmu dla każdej obserwacji jest średnia długość ścieżki, na której została ona odizolowana, uśredniona po wszystkich drzewach w lesie. Długość ścieżki jest liczona jako liczba krawędzi od korzenia do liścia, w którym znalazła się obserwacja. Na podstawie tej średniej obliczany jest współczynnik anomalii [9].

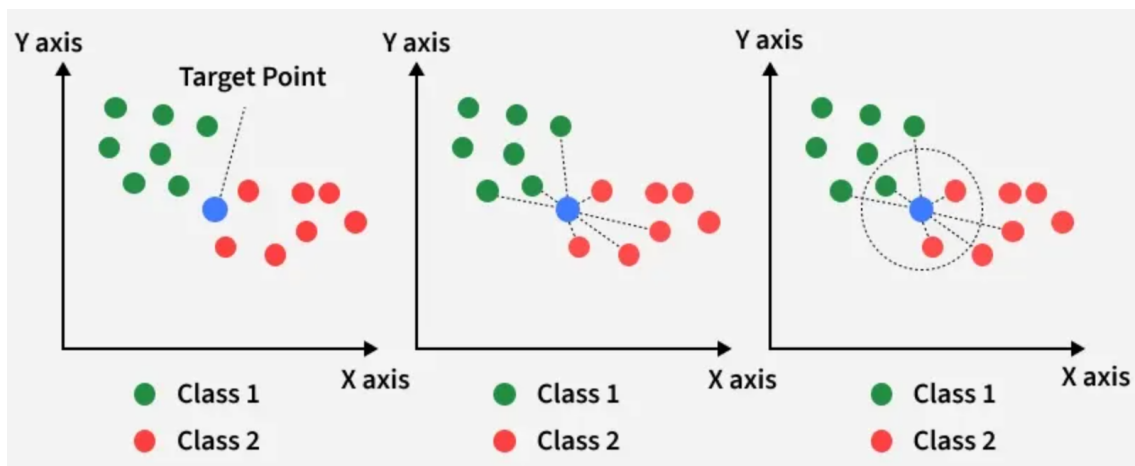
• Local Factor Outlier

Local Outlier Factor (LOF) reprezentuje metodę detekcji anomalii, której idea stanowi analiza lokalnych właściwości zbioru danych. Podejście to różni się fundamentalnie od metod globalnych (Isolation Forest) tym, że każda próbka podlega ewaluacji w kontekście swojego bezpośredniego otoczenia, a nie w relacji do całości zbioru danych. Obserwacja otrzymuje ocenę nie tylko ze względu na swoje umiejscowienie w wielowymiarowej przestrzeni atrybutów. Kluczowe znaczenie ma tutaj zestawienie jej lokalnej gęstości z analogicznymi charakterystykami punktów sąsiadujących. Anomalia wcale nie musi lokować się na peryferiach zbioru danych. Rzeczywiste zbiory danych zawierają anomalie w regionach, które z perspektywy globalnej nie budzą podejrzeń. Te punkty wykazują jednak istotnie zredukowaną koncentrację w swoim lokalnym sąsiedztwie, co stanowi właściwą podstawę ich identyfikacji. LOF realizuje detekcję poprzez sekwencyjną estymację gęstości lokalnej dla poszczególnych obserwacji. Następnie przeprowadza się porównanie otrzymanych wartości z odpowiednimi punktami wokół badanego punktu. Taki mechanizm umożliwia wykrywanie anomalii kontekstowych, których identyfikacja metodami globalnymi okazuje się niemożliwa lub znacząco utrudniona. Wartość

współczynnika LOF odzwierciedla stopień, w jakim lokalna gęstość danej obserwacji odbiega od gęstości typowej dla jej otoczenia.

Aby zrozumieć działanie LOF, warto wyjaśnić kilka kluczowych pojęć [10]:

- **k-sąsiedztwo (ang. K-Nearest Neighbor (KNN))**: KNN stanowi technikę wyszukiwania wykorzystywaną przez LOF do identyfikacji k najbliższych sąsiadów każdego punktu. Wartość k jest parametrem algorytmu - im mniejsze k , tym bardziej lokalna analiza. W przeciwieństwie do klasycznego zastosowania KNN w zadaniach klasyfikacyjnych, LOF wykorzystuje tę metodę wyłącznie do wyznaczenia lokalnego otoczenia punktów, nie do predykcji etykiet klas [11].



RYSUNEK 3.2: Ilustracja zasady działania KNN [11]

- **Lokalna gęstość**: Algorytm oblicza, jak “gęsto upakowane” jest bezpośrednie otoczenie każdego punktu. Obszar o wysokiej gęstości to taki, gdzie punkty leżą blisko siebie, podczas gdy w obszarze o niskiej gęstości punkty są rozproszone.
- **Współczynnik LOF**: Ostateczny wskaźnik to stosunek gęstości punktu do średniej gęstości jego sąsiadów.

Interpretacja wyniku jest intuicyjna [10]:

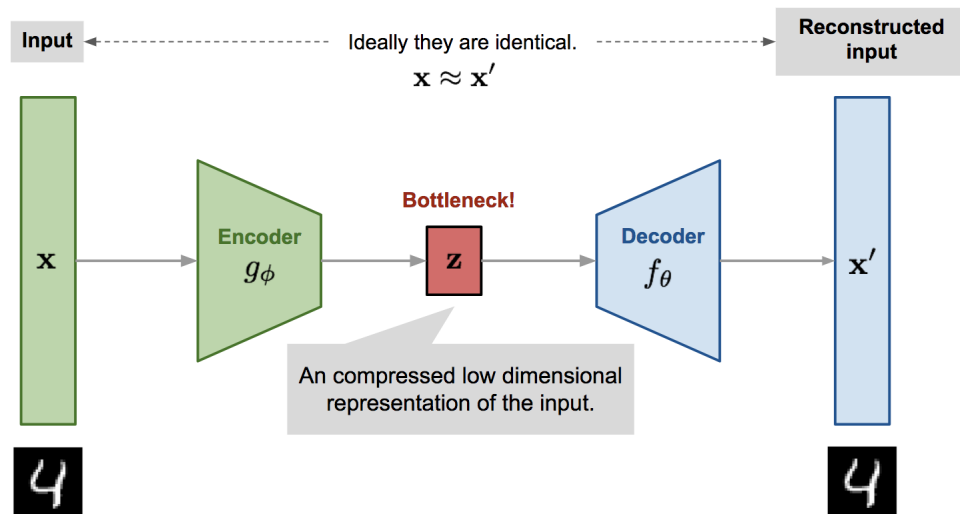
- **LOF ≈ 1** : Gęstość punktu jest podobna do gęstości jego sąsiadów - punkt normalny
- **LOF > 1** : Punkt ma niższą gęstość niż jego otoczenie - potencjalna anomalia
- **LOF < 1** : Punkt ma wyższą gęstość niż sąsiedzi - punkt wewnątrz skupiska

• Autoencoder

Autoencoder stanowi specyficzną architekturę sieci neuronowej, która realizuje zadanie uczenia nienadzorowanego poprzez odtwarzanie na wyjściu danych wejściowych. Sieć ta składa się z dwóch fundamentalnych części: kodera, który dokonuje kompresji wejścia do postaci reprezentacji ukrytej o mniejszym wymiarze, oraz dekodera, którego zadaniem jest rekonstrukcja oryginalnych danych z tej skompresowanej reprezentacji. Proces uczenia polega na minimalizacji błędu rekonstrukcji, co zmusza model do wychwycenia najistotniejszych wzorców i cech charakteryzujących dane treningowe [12]. W kontekście wykrywania anomalii, Autoencodery

są szkolone wyłącznie na danych uważanych za normalne. Model uczy się efektywnie rekonstruować w jaki sposób powinna wyglądać dana gdyby była częścią normy. Gdy na wejściu pojawi się obserwacja anormalna, która znacząco odbiega od tego wzorca, proces rekonstrukcji zazwyczaj zawodzi, prowadząc do wysokiego błędu rekonstrukcji. Błąd ten jest mierzony i służy jako miara anomalii, im większy błąd tym większa anomalia.

Architektura Autoencodera jest z założenia symetryczna. Warstwy kodera stopniowo redukują wymiarowość danych, aż do osiągnięcia wąskiego gardła (ang. bottleneck), czyli warstwy reprezentacji ukrytej, która ma najmniejszą liczbę neuronów (rys. 3.3). Następnie warstwy dekodera próbują odwrócić ten proces. Kluczowym elementem jest właśnie to wąskie gardło, które wymusza na sieci naukę odtwarzania reprezentacji danych wejściowych na podstawie części skompresowanej [13].



RYSUNEK 3.3: Ilustracja architektury Autoencodera przedstawiona na podstawie wejściowej danej rysunku liczby „4” i jej odtworzenia. W kontekście wykrywania anomalii Autoenkoder uczy się odtwarzać poprawne przykłady danych w tym przypadku cyfrę „4”. Jeżeli rekonstrukcja istotnie różni się od obrazu wejściowego, może to wskazywać, że na wejściu pojawiła się anomalia, czyli obiekt różniący się od danych, na których model był trenowany [13]

Rozdział 4

Implementacja Isolation Forest

Do implementacji modelu użyta została biblioteka "Scikit-learn Machine Learning in Python". Biblioteka oprogramowania została stworzona w 2007 roku i rozwijana przez kolejne lata jako projekt "kodu otwartego" (ang. open source) oznacza to, że kod jest opublikowany w całości do darmowego wglądu i użytku [14].

W badaniach do implementacji IF zostały użyte funkcje z sekcji "ensemble". Forma algorytmu została zaimplementowana w jego podstawowej wersji, dostępnej w bibliotece. Scikit-learn stanowi jedną z najpopularniejszych, napisanych w języku Python, i dedykowanych uczeniu maszynowemu. Biblioteka oferuje kompleksowy zestaw narzędzi do implementacji algorytmów uczenia nadzorowanego, nienadzorowanego oraz półnadzorowanego. W kontekście wykrywania anomalii scikit-learn udostępnia szereg algorytmów opartych na różnych podejściach metodologicznych, umożliwiających identyfikację obserwacji odstających w zbiorach danych.

W implementacji modelu użyte zostały podstawowe parametry, które umożliwiają jak najlepsze dopasowanie pracy algorytmu do danych z ruchu sieciowego. Są nimi: `n_estimators`, `max_samples`, `contamination`, `max_features`, `bootstrap`, `n_jobs`, `random_state`, `verbose`, `warm_start` (rys. 4.1). Mimo że istnieje możliwość dodania większej ilości parametrów poprzez własną edycję biblioteki. Tutaj badania skupiły się na przedstawieniu pracy i dostosowania klasycznej, szeroko używanej wersji. Aby jak najlepiej zrozumieć dobrane wartości do modelu Isolation Forest w rozdziale skupiającym się na opisie testów, wpierw warto przybliżyć znaczenie i funkcje poszczególnych charakterystyk [8], [14]:

- **n_estimators:**

Określa liczbę drzew tworzonych w lesie izolacyjnym. Każde drzewo izolacyjne analizuje próbki niezależnie i generuje własną informację o tym, jak szybko dana obserwacja została odizolowana od pozostałych. Większa liczba drzew zwiększa liczbę niezależnych statystyk dotyczących tego, jak każda próbka odstaje od wzorca normalnego. W konsekwencji uzyskuje się bogatszy obraz rozkładu długości ścieżek izolacji, co zmniejsza wpływ losowości pojedynczych podziałów cech na ostateczną ocenę anomalii. Zwiększenie liczby drzew zwykle poprawia stabilność i powtarzalność wyników, ale jednocześnie podnosi zapotrzebowanie na zasoby obliczeniowe i pamięć.

W podejściu mającym jak najlepsze dopasowanie pracy modelu do analizy danych sieciowych charakteryzującą się dużą ilością cech w jednej próbce. Podjęto decyzję o zwiększeniu ilości drzew w lesie z wzorcowych 100 do 250. W fazie testowania powoduje to zwiększenie precyzji modelu, pozostając pod rekomendowanym sufitem 300 [15].

- **max_samples:**

Określa maksymalną liczbę próbek wykorzystywanych do budowy pojedynczego drzewa izolacyjnego. Dla każdego drzewa losowo wybierany jest podzbiór danych, na podstawie którego tworzona jest struktura podziałów. Większa liczba próbek prowadzi do budowy głębszych i bardziej rozbudowanych drzew, co skutkuje uwzględnieniem szerszego kontekstu danych. Zmniejszenie wartości `max_samples` ogranicza rozmiar poszczególnych drzew, co przekłada się na krótszy czas uczenia oraz mniejsze zapotrzebowanie na pamięć. Jednocześnie redukowana jest podatność modelu na losowe wariacje wynikające z przypadkowego doboru cech [14].

W implementacji `scikit-learn` domyślną wartością jest `'auto'`, odpowiadające `min(256, n_samples)`. Jednak przy dużych zbiorach danych normalnego ruchu sieciowego zwiększenie tej wartości powoduje, że każde drzewo jest trenowane na znacznie większej próbce danych, co poprawia reprezentatywność statystyczną lasu oraz stabilność detekcji anomalii, choć jednocześnie zwiększa koszt obliczeniowy [16]. Użycie znacznie większej wartości 8192 próbek pozwoliło zwiększyć skuteczność modelu, z uzasadnionego powodu zwiększenia jego pola widzenia.

- **contamination:**

Określa oczekiwany udział obserwacji anormalnych w analizowanym zbiorze danych. Wartość ta nie wpływa bezpośrednio na proces budowy drzew izolacyjnych, lecz odgrywa kluczową rolę na etapie interpretacji wyników oraz ustalania granicy decyzyjnej pomiędzy danymi normalnymi a anomaliami [14].

Powód dla którego zdecydowano obrać parametr o wartości 0,01 jest powiązany z założeniem, że anomalie są rzadkim zdarzeniem. Model szuka pojedynczych sygnatur odbiegających od normy działania. W praktyce ustawienie zmniejszą również dla ograniczenia FP, które mocno ingerowały w poprawną interpretację danych.

- **max_features:**

Określa liczbę cech wykorzystywanych podczas budowy pojedynczego drzewa izolacyjnego. W przypadku zbiorów danych o dużej liczbie cech, z których część może nie wносить istotnej informacji lub wręcz negatywnie wpływać na proces detekcji, możliwe jest ograniczenie liczby cech branych pod uwagę w każdym drzewie. Ustawienie wartości `max_features` na poziomie 1,0 skutkuje wykorzystaniem pełnego zestawu cech dostępnych w zbiorze danych. Obniżenie tej wartości, na przykład do 0.8, oznacza że w procesie budowy drzewa losowo pomijane jest 20% cech [14].

Szukając anomalii w ruchu sieciowym założono, że nieznany atak może dojść z każdej strony. Z tego powodu zdecydowano się nie ograniczać cech podczas procesu uczenia. Chcąc wykrywać nieznane ataki nie jest możliwe określenie jaki procent cech z danych ruchu normalnego, będzie mniej użyteczny.

- **bootstrap:**

posiada dwa stany `"true"` i `"false"`, stan aktywacji i dezaktywacji losowania próbek z powtórzeniami. W przypadku ustawienia wartości `false` mechanizm losowania odbywa się bez zwracania, co oznacza że każda próbka może zostać wykorzystana w danym drzewie tylko jednokrotnie. Ustawienie wartości `true` aktywuje losowanie z powtórzeniami, w wyniku czego te same próbki mogą pojawić się wielokrotnie podczas budowy pojedynczego drzewa, a inne zostać pominięte [14].

Parametr ustawiony na "false" wynika z chęci ograniczenia nakładania się wyników w drzewach. Losowe powtórzenia mogą wpłynąć pozytywnie na jakość detekcji, jednak podjęto decyzję na ograniczenie tej funkcji, aby zmniejszyć powiązanie jakości wyników z pojedynczo szczególnie dobraniem powtórzeń z testowanego zbioru.

- **n_jobs:**

określa bezpośrednio dopasowanie prac do możliwości obliczeniowego. Poprzez zwiększanie wartości umożliwia się równoległy proces obliczeniowy dla x prac równocześnie [14].

W przypadku badań nie było konieczne przyspieszenie pracy modelu ze względu na badanie potencjału wyników zamiast prędkości obliczeń i tępa zwracania wyniku.

- **random_state:**

określa wartość początkową generatora liczb losowych, nazywaną ziarnem losowości (ang. seed). Ustawienie tej wartości powoduje utrzymanie stałego stanu losowego podczas działania algorytmu, mimo jego losowego charakteru. Dzięki temu proces budowy drzew, dobór próbek oraz wybór cech przebiegają w identyczny sposób przy każdym uruchomieniu modelu. Umożliwia dokładną replikację uzyskanych wyników, co ma istotne znaczenie w analizie jakości dopasowania algorytmu oraz w porównywaniu różnych konfiguracji parametrów i charakterystyk [14].

Wartość wstawiona 33, nie posiada głębszej argumentacji poza faktem, że stała wartość umożliwia łatwiejszą pracę nad parametrami. Dobrano liczbę 33 ze względów bliżej nie określonych i z konieczności wstawienia losowej wartości.

- **verbose:**

Posiada dwa stany "true" i "false", stan aktywacji i dezaktywacji wyświetlania szczegółów procesu. Podczas przebiegu testów dodatkowe informacje generowane przez model są zaznaczane kolorem czerwonym (rys. 4.1).

Stan aktywacji pozwolił na uzyskanie większej ilości danych i wglądu w pracę modelu. Nie wpływa on w żaden sposób na charakterystykę.

- **warm_start:**

Posiada dwa stany "true" i "false" określa, czy algorytm podczas ponownego uruchomienia wykorzystuje wcześniej utworzoną strukturę modelu. Przy ustawieniu wartości „false” każda iteracja uczenia rozpoczyna się od zera, a las izolacyjny budowany jest niezależnie od wcześniejszych obliczeń. Ustawienie wartości „true” powoduje zachowanie wcześniej utworzonych drzew i ich uzupełnianie o kolejne elementy. Pozwala to stopniowo rozwijać model bez pełnej rekonstrukcji lasu, jednak wprowadza zależność wyników od wcześniejszego przebiegu uczenia [14].

Ustawiono parametr na "false", ponieważ pozwala to na uniknięcie przepisu artefaktów. Podczas dopracowywania charakterystyk modelu każda iteracja treningu kolejnej propozycja modelu różniła się znacząco zasadą dobierania wartości. Oznacza to że tworzony las miał zadanie za każdym razem powstawać od samego początku, bez jakiegokolwiek powiązania z poprzednią generacją.

Isolation Forest

```
In [3]: IF = IsolationForest(
        n_estimators=250,
        max_samples=8192,
        contamination= 0.01,
        max_features=1.0,
        bootstrap=False,
        n_jobs=None,
        random_state=33,
        verbose=1,
        warm_start=False
    )
    params_IF = '\n'.join(f'{k}: {v}' for k, v in IF.get_params().items())
    funs.log("Parameters IF:\n" + params_IF)

    IF.fit(df_training)
```

Parameters IF:
bootstrap: False
contamination: 0.01
max_features: 1.0
max_samples: 8192
n_estimators: 250
n_jobs: None
random_state: 33
verbose: 1
warm_start: False

[Parallel(n_jobs=1)]: Done	1 out of 1	elapsed:	3.2s finished
[Parallel(n_jobs=1)]: Done	49 tasks	elapsed:	0.8s
[Parallel(n_jobs=1)]: Done	199 tasks	elapsed:	3.0s
[Parallel(n_jobs=1)]: Done	250 out of 250	elapsed:	3.8s finished

```
Out[3]: IsolationForest(contamination=0.01, max_samples=8192, n_estimators=250,
                        random_state=33, verbose=1)
```

RYSUNEK 4.1: Wycinek kodu ilustrujący implementację modelu IF, mający na celu przedstawienie składni

Rozdział 5

Implementacja Local Outlier Factor

Podobnie jak w przypadku Isolation Forest, do implementacji Local Outlier Factor została użyta biblioteka Scikit-learn. W przypadku LOF z sekcji „neighbors”. Dzięki użyciu zasobów z tej samej biblioteki, różnice w składni nie różni się znacząco od IF. Widoczne różnice są natomiast w zasadzie działania. Mimo że LOF jest również modelem uczenia maszynowego nienadzorowanego do wykrywania anomalii, używa on zupełnie innej metody.

Porównując lokalną gęstość danej próbki z lokalnymi gęstościami jej sąsiadów, można zidentyfikować próbki, które mają wyraźnie mniejszą gęstość niż ich sąsiedzi. Takie próbki uznaje się za obserwacje odstające (outliery). Można to uzyskać, dopasowując określone parametry modelu do danych, na których będzie przeprowadzany trening. W badaniu myślą przewodnią było również użycie klasycznej implementacji algorytmu LOF, dlatego definiowane parametry modelu zostały wybierane z myślą o zachowaniu balansu między adaptacją modelu do ruchu sieciowego a utrzymaniem charakterystycznych zachowań LOF (rys. 5.1) [17]:

- **n_neighbors:**

określa liczbę najbliższych obserwacji, które są brane pod uwagę przy estymacji lokalnej gęstości punktu. Wartość ta ma kluczowe znaczenie, ponieważ definiuje skalę lokalności analizy. Zbyt mała liczba sąsiadów może prowadzić do nadwrażliwości modelu na szum, natomiast zbyt duża powoduje uśrednianie gęstości i utratę zdolności wykrywania subtelnych anomalii [17].

Wartość 50 jest dwa razy większa od wartości domyślnej. Powodem dla którego zwiększono tą wartość jest próba akomodacji dużej ilości próbek w danych ruchu sieciowego przy zachowaniu naturalnego zachowania modelu do badań.

- **algorithm:**

Określa wybór metody wyszukiwania najbliższych sąsiadów w przestrzeni cech. Ustawienie auto pozwala algorytmowi samodzielnie dobrać najbardziej efektywną strategię obliczeniową w zależności od charakterystyki danych, takich jak ich liczność czy wymiarowość. Nie wpływa on bezpośrednio na wynik detekcji, lecz na efektywność obliczeń. Do manualnego wyboru dostępne są trzy algorytmy [18], [19], [11]:

- *Ball Tree* - polega na prostym, liniowym obliczaniu odległości pomiędzy punktem zapytania a każdym innym punktem w zbiorze. Nie buduje żadnej struktury pomocniczej i zawsze przegląda wszystkie obserwacje, co prowadzi do złożoności czasowej proporcjonalnej do liczby punktów ($O(N \times d)$, gdzie d to liczba wymiarów). Dzięki temu wynik

jest dokładny, lecz przy dużych zbiorach danych obliczanie odległości dla każdej pary punktów jest kosztowne i mało skalowalne.

- *KD Tree* - To i drzewo binarne, w którym każdy węzeł reprezentuje punkt k-wymiarowy w przestrzeni. Drzewo rekurencyjnie dzieli zbiór danych na dwie połowy wzdłuż osi charakteryzującej się największą wariancją punktów danych. Taka struktura umożliwia efektywne przeszukiwanie oraz eliminację punktów danych znacznie oddalonych od punktu docelowego, redukując liczbę punktów wymagających rozpatrzenia podczas wyszukiwania sąsiadów.
- *Brute-force search* - polega na prostym, liniowym obliczaniu odległości pomiędzy punktem zapytania a każdym innym punktem w zbiorze. Nie buduje żadnej struktury pomocniczej i zawsze przegląda wszystkie obserwacje, co prowadzi do złożoności czasowej proporcjonalnej do liczby punktów ($O(N \times d)$, gdzie d to liczba wymiarów). Dzięki temu wynik jest dokładny, lecz przy dużych zbiorach danych obliczanie odległości dla każdej pary punktów jest kosztowne i mało skalowalne

Dzięki możliwości wyboru automatycznego doboru algorytmu, uzyskano możliwość dobrania do właściwości środowiska. Jest to wartość domyślna, którą pozostawiano.

- **leaf_size:**

Określa rozmiar liścia przekazywany do struktur wyszukiwania najbliższych sąsiadów (BallTree lub KDTree) używanych przy estymacji lokalnej gęstości. Wartość ta nie wpływa na sam wynik wykrywania anomalii, jednak ma znaczący wpływ na efektywność obliczeń i ilość pamięci potrzebnej do przechowywania drzewa indeksującego [17].

Pozostawiono domyślną wartość 30 ze względu na wcześniejszą decyzję o doborze trybu automatycznego doboru algorytmu. Ze względu na wpływ jedynie na efektywność obliczeń pozostawiono tę wartość tak jak została ona dobrana w dokumentacji [17].

- **metric:**

Określa, w jaki sposób mierzone są odległości między obserwacjami podczas wyszukiwania najbliższych sąsiadów i estymacji lokalnej gęstości. Posiada wiele różnych metryk do obliczania odległości. Najczęściej używana jest metryka "minkowski" określająca całą rodzinę metryk [20].

- **p:**

Parametr mający znaczenie tylko w przypadku wyboru metryki „minkowski”, określa którą konkretnie metrykę z rodziny wybierzemy. Wzór na określanie odległości w rodzinie Minkowskiego ma dosyć prostą zasadę działania, którą można zilustrować za pomocą odległości w drugim wymiarze [17].

$$d(x, y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p} \quad (5.1)$$

gdzie: $x, y \in R^n$ – punkty w przestrzeni n -wymiarowej, $p \geq 1$ – parametr metryki.

- **$p=1$** (*Manhattan/Taksówkowa*) – suma bezwzględnych różnic współrzędnych. W przestrzeni dwuwymiarowej odpowiada odległości mierzonej wzdłuż osi układu współrzędnych, analogicznie do przemieszczania się po siatce ulic:

$$d(x, y) = \sum_{i=1}^n |x_i - y_i| = |x_1 - y_1| + |x_2 - y_2| + \dots + |x_n - y_n| \quad (5.2)$$

- $p=2$ (*Euklidesowa*) – klasyczna odległość w linii prostej, oparta na twierdzeniu Pitagorasa. Stanowi najbardziej intuicyjną miarę dystansu w przestrzeni fizycznej:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots + (x_n - y_n)^2} \quad (5.3)$$

- $p \rightarrow \infty$ (*Czebyszewa/Maksimum*) – w granicy, gdy p dąży do nieskończoności, odległość Minkowskiego redukuje się do maksymalnej bezwzględnej różnicy między współrzędnymi:

$$d(x, y) = \max_{i=1, \dots, n} |x_i - y_i| \quad (5.4)$$

W wielu przeglądach technik wykrywania anomalii podkreśla się, że techniki oparte na najbliższych sąsiadach, szczególnie z wykorzystaniem odległości euklidesowej, są naturalnym i powszechnie stosowanym sposobem modelowania lokalnej gęstości danych w kontekście wykrywania punktów odstających. Chcąc utrzymać klasyczną interpretację modelu jak i poprawną implementację zastosowano wartość p równą 2, czyli użyto klasyczne euklidesowe podejście sugerowane przez bibliotekę [17].

- **metric_params:**

Umożliwia przekazanie dodatkowych argumentów do funkcji metryki w formie słownika klucz-wartość. Struktura ta pozwala na elastyczną konfigurację metryk wymagających parametrów wykraczających poza standardową definicję [17].

Wykorzystano brak wartości pod nazwą „none” ze względu na klasyczną implementacji. Odzworowane w ten sposób zachowanie LOF staje się bardziej przewidywalne i bliższe domyślnemu działaniu algorytmu.

- **contamination:**

Podobnie jak w IF, określa oczekiwany udział obserwacji anormalnych w analizowanym zbiorze danych. Wartość ta nie wpływa bezpośrednio na proces. Znaczenie tego parametru jest kluczowe podczas interpretacji wyników [17].

Wybór wartości „auto” oznacza wartość 0,01 powielając założenia z IF, czyli z założeniem że anomalie są rzadkim zdarzeniem.

- **novelty:** Określa schemat operacyjny algorytmu LOF, definiując fundamentalną różnicę między detekcją outlierów a detekcją nowości w kontekście uczenia maszynowego. Posiada dwa tryby [17]:

- *True* - algorytm przyjmuje założenie, że zbiór treningowy składa się wyłącznie z obserwacji reprezentatywnych dla rozkładu normalnego, a jego zadaniem jest detekcja anomalii w nowych, nieznanach danych.
- *False* - algorytm zakłada, że zbiór treningowy może zawierać obserwacje odstające i wykonuje ich identyfikację w obrębie tego samego zbioru.

Istotnym momentem dla badań jest określenie charakterystyki na wartość „True”. Zgodnie z założeniami, ustalono chęć wykrywania anomalii (ataków) w ruchu sieciowym na podstawie uczenia modelu na ruchu normalnym. Oznacza to, że możliwość ustawienia novelty na „True” jest idealnym dla założeń pracy.

- **n_jobs:**

Podobnie jak w IF, określa bezpośrednio dopasowanie prac do możliwości obliczeniowego. Poprzez zwiększanie wartości umożliw się równoległy proces obliczeniowy dla x prac równocześnie [17].

Powielając schemat z IF, nie było konieczne przyspieszenie pracy modelu ze względu na badanie potencjału wyników zamiast tępa zwracania wyniku.

Local Outlier Factor

```
In [4]: LOF= LocalOutlierFactor(  
        n_neighbors=50,  
        algorithm='auto',  
        leaf_size=30,  
        metric='minkowski',  
        p=2,  
        metric_params=None,  
        contamination='auto',  
        novelty=True,  
        n_jobs=None  
    )  
    params_LOF = '\n'.join(f'{k}: {v}' for k, v in LOF.get_params().items())  
    funs.log("Parameters LOF:\n" + params_LOF)  
  
    LOF.fit(df_training)
```

Parameters LOF:
algorithm: auto
contamination: auto
leaf_size: 30
metric: minkowski
metric_params: None
n_jobs: None
n_neighbors: 50
novelty: True
p: 2

```
Out[4]: LocalOutlierFactor(n_neighbors=50, novelty=True)
```

RYSUNEK 5.1: Wycinek kodu ilustrujący implementację LOF, mający na celu przedstawienie składni

Rozdział 6

Implementacja Autoencodera

6.1 Użyte zasoby

Autoencoder nie jest typowym modelem uczenia maszynowego. W jego implementacji zastosowane zostały rozwiązania z sieci neuronowych. Przez to unikalne podejście w porównaniu do LOF i IF nie został on zawarty w Scikit-learn. Implementacja Autoencodera została zaciągnięta z popularnej biblioteki PyTorch. W implementacji konieczne było zdefiniowanie każdej warstwy, jak i podstawowej architektury. Autoencoder nie został zdefiniowany w bibliotece w formie gotowego do użycia modelu, lecz konieczne było jego przygotowanie od podstaw. Ze względu na analityczne podejście do testowania podstawowej zasady działania modelu, implementowana architektura została również zaprojektowana z myślą o najlepszym reprezentowaniu zasady działania modelu, a nie o skrupulatnym dopasowaniu go do danych testowych.

6.2 Implementacja

Proces implementowania modelu różni się znacząco od dwóch poprzednich modeli. Nie posiada on od razu zdefiniowanej funkcji „fit()”. Biblioteka PyTorch dostarcza tylko narzędzia do implementacji sieci neuronowych. Zaimplementowane moduły z biblioteki to:

- torch.nn - zawierający komponenty oraz narzędzia, takie jak "DataLoader" i "TensorDataset"
- nn.Module - klasa bazowa, po której konieczna jest samodzielna definicja architektury (nn.Sequential dziedziczy po nn.Module)
- nn.Sequential - narzędzie służące do sekwencyjnego łączenia warstw sieci neuronowej w jedną spójną strukturę
- nn.Linear - narzędzie realizujące transformację liniową, polegającą na mapowaniu wektora wejściowego na wektor wyjściowy poprzez mnożenie przez macierz wag. Operacja ta definiuje połączenia między neuronami kolejnych warstw oraz przypisuje im wagi uczone w trakcie treningu sieci.
- nn.ReLU - narzędzie realizujące nieliniową transformację, która działa na wyjściu warstwy liniowej. Jej działanie polega na przepuszczaniu dalej jedynie wartości dodatnich, przy jednoczesnym zerowaniu wartości ujemnych.

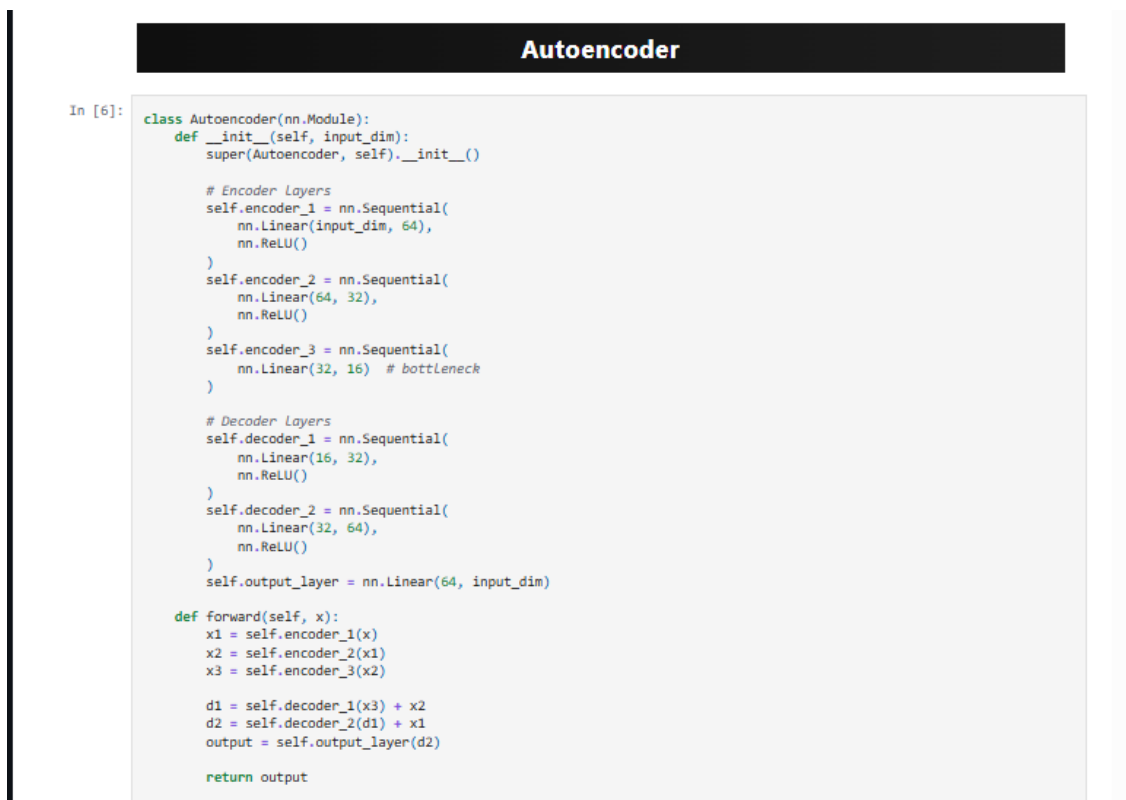
6.3 Użyta architektura

Zaimplementowana architektura Autoenkodera została zaprojektowana w klasycznym symetrycznym podejściu typu encoder-decoder, powszechnie stosowanym w detekcji anomalii. To podejście konstrukcyjne miało na celu jak najczystsze odwzorowanie modelu zamiast maksymalizacji jakości predykcji. Strategia ta pozwala na czytelniejszą interpretację wniosków z testów jak i ocenę potencjału technologii [21].

Pierwszą częścią w definicji klasy (rys. 6.1) jest część kodująca (ang. encoder). Składa się ona z trzech warstw liniowych, których funkcją jest redukcja wymiaru przestrzeni cech z początkowych 64 przez 32 do 16. Stopniowa redukcja wymiarowości jest klasycznym rozwiązaniem w autoenkoderach, a jej celem jest zmuszenie procesu kompresji informacji. Bottleneck, czyli warstwa o najniższym wymiarze, stanowi reprezentację danych wejściowych i jest najważniejsza w procesie uczenia wzorców. W trzech warstwach enkodera zastosowano funkcję aktywacji ReLu, która również jest powszechnie używana w standardowym modelu. Ta funkcja wprowadza nieliniowość do modelu, co umożliwia generowanie bardziej złożonych zależności pomiędzy cechami, przy jednoczesnym zachowaniu stabilności procesu uczenia.

Część dekodująca (ang. decoder) została zaprojektowana w symetryczny sposób do części kodującej i odpowiada za rekonstrukcję danych z przestrzeni skompresowanej/ukrytej z powrotem do wymiaru wyjścia. Symetryczność warstwy charakteryzuje się tym samym przejściem co poprzedni etap, ale w odwrotnej kolejności: z 16 przez 32 do 64, a następnie do oryginalnego rozmiaru wyjścia. To podejście jest charakterystyczne dla klasycznych Autoencoderów i ułatwia interpretację działania modelu jako procesu kompresji i dekompresji [22].

W końcowym etapie zanotowane zostało rozwiązanie połączenia rezyduального. Połączenie polega na bezpośrednim dodaniu wyjścia z wcześniejszej warstwy enkodera do odpowiadającej jej warstwy dekodera, z pominięciem warstwy o mniejszym wymiarze. Celem tego rozwiązania jest ograniczenie strat informacji, które pojawiają się podczas redukcji wymiarowości. W klasycznym autoenkoderze cała informacja o danych wejściowych musi zostać odtworzona wyłącznie z reprezentacji w warstwie bottleneck. W przypadku danych o wysokiej liczbie cech (jak dane sieciowe) może to prowadzić do nadmiernego uproszczenia i pogorszenia rekonstrukcji. Zastosowany mechanizm połączeń rezydualnych daje dekodownikowi możliwość korzystania zarówno z informacji zawartej w skompresowanej reprezentacji, jak i z bardziej szczegółowych informacji zawartych w wcześniejszych etapach. Z punktu widzenia celu projektowanego modelu to rozwiązanie stało się niezwykle pomocne w roli stabilizującej działanie modelu. Warto podkreślić, że zastosowane połączenia rezydualne mają charakter prosty i są ograniczone do odpowiadających sobie warstw enkodera i dekodera. Nie stanowią one próby implementacji zaawansowanych architektur rezydualnych znanych z głębokich sieci neuronowych, lecz są wymuszonym kompromisem pomiędzy czytelnością modelu a poprawą stabilności rekonstrukcji [22].



RYSUNEK 6.1: Wycinek kodu ilustrujący implementację Autoencodera, mający na celu przedstawienie zasady implementacji w badaniu

6.4 Proces uczenia

Trening Autoenkodera również wymagał implementacji. Zastosowano klasyczne podejście uczenia w sposób iteracyjny. Tak jak wymagają tego założenia, trening odbył się wyłącznie na danych normalnego ruchu sieciowego, bez wykorzystania etykiet. Celem uczenia, w odróżnieniu od IF i LOF, nie jest klasyfikacja obserwacji, lecz minimalizacja błędu rekonstrukcyjnego.

Uczenie jest realizowane w pętli epok (rys 6.2), gdzie jedno pełne przejście pętli oznacza przejście przez cały zbiór treningowy. Dane wejściowe zostały podzielone na mniejsze części (ang. batch), co pozwala na stabilniejsze oraz bardziej efektywne obliczenia. Dla każdego batcha wykonano „forward pass”, czyli przejście danych kolejno przez warstwy enkodera, bottleneck i dekodera. Następnie obliczona została funkcja strat, która w tym przypadku mierzy różnicę pomiędzy danymi początkowymi a ich rekonstrukcją.

W kolejnym kroku wykonywany jest backward pass, czyli propagacja wstecz. Na podstawie obliczeń z funkcji strat wyznaczane są gradienty wag poszczególnych warstw, które są wykorzystywane do optymalizacji parametrów w warstwach, w celu minimalizacji błędów rekonstrukcyjnych w kolejnym przejściu przez pętlę. Każde przejście jest monitorowane i wyświetlane zostaje współczynnik loss. W loss zostaje akumulowana wartość funkcji strat z pojedynczego przejścia. Im mniejsza strata (ang. loss) tym lepiej model zrekonstruował dane.

Intuicyjne podsumowanie procesu uczenia Autoenkodera można przedstawić jako sekwencję pięciu następujących po sobie kroków. W pierwszym kroku model otrzymuje porcję danych treningowych reprezentujących normalny ruch sieciowy. W drugim kroku dane te poddawane są procesowi kompresji w części kodującej, a następnie rekonstrukcji w części dekodującej, czego wynikiem jest

próba odtworzenia danych wejściowych. W trzecim kroku obliczana jest wartość funkcji straty, która określa stopień błędu popełnionego przez model podczas rekonstrukcji danych. Następnie, w czwartym kroku, na podstawie obliczonego błędu wykonywana jest korekta wag sieci neuronowej poprzez propagację wsteczną i aktualizację parametrów modelu. W piątym i końcowym kroku prezentowane są wyniki aktualnego przebiegu uczenia, po czym cała procedura jest ponawiana dla kolejnych porcji danych oraz następnych epok treningowych [21].

```
num_epochs = 20

for epoch in range(num_epochs):
    total_loss = 0
    for batch in dataloader:
        batch_X = batch[0]

        # Forward pass
        outputs = model_A_1(batch_X)
        loss = criterion(outputs, batch_X)

        # Backward pass and optimize
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    total_loss += loss.item()

    print(f"Epoch [{epoch+1}/{num_epochs}], Loss: {total_loss:.4f}")
```

Epoch [1/20], Loss: 207.2547
Epoch [2/20], Loss: 176.1922
Epoch [3/20], Loss: 183.1357
Epoch [4/20], Loss: 189.4172
Epoch [5/20], Loss: 187.9328
Epoch [6/20], Loss: 158.8084
Epoch [7/20], Loss: 95.0605
Epoch [8/20], Loss: 92.0766
Epoch [9/20], Loss: 108.1528
Epoch [10/20], Loss: 104.7567
Epoch [11/20], Loss: 54.2499
Epoch [12/20], Loss: 57.9004
Epoch [13/20], Loss: 47.8503
Epoch [14/20], Loss: 57.6764
Epoch [15/20], Loss: 34.4429
Epoch [16/20], Loss: 47.7204
Epoch [17/20], Loss: 46.0496
Epoch [18/20], Loss: 32.0739
Epoch [19/20], Loss: 29.1863
Epoch [20/20], Loss: 41.6977

RYSUNEK 6.2: Wycinek kodu ilustrujący implementację fazy treningowej Autoencodera, mający na celu przedstawienie zasady implementacji w badaniu

Rozdział 7

Dane i przygotowanie środowiska

7.1 Zbiór danych CICIDS2017

Do wykonania badań wykorzystano zbiór „Intrusion detection evaluation dataset (CIC-IDS2017)”, który jest powszechnie stosowanym i ogólnodostępnym zbiorem ruchu sieciowego Canadian Institute for Cybersecurity (CIC) działającego przy University of New Brunswick (UNB) [23], [24]. Zbiór ten został wybrany ze względu na realistyczne odwzorowanie ruchu sieciowego, oraz fakt, że jest on często spotykany i jednym z najczęściej wykorzystywanych źródeł danych w badaniach wykrywania ataków sieciowych.

Baza danych posiada normalny ruch sieciowy, jak i szeroki zakres symulacji ataków. Uwzględniono najczęstsze ataki oparte na raporcie McAfee z 2016 r., takie jak ataki internetowe, ataki siłowe, ataki DoS, ataki DDoS, infiltracja, ataki typu Heartbleed, ataki typu bot i skanowanie omówione w tym zestawie danych [23], [24]. Rejestracja danych została opracowana jako rzeczywiste przepływy sieciowe (ang. network flows) każdy z przepływów posiada szczegółowo opisane cechy, takie jak charakterystyki pakietów, kierunek komunikacji czy znaczniki czasowe. Choć w badaniach modele uczenia nienadzorowanego są trenowane tylko i wyłącznie na danych bez oznaczeń, które z flow są częścią ataku, zbiór danych posiada etykiety złośliwych przepływów. Pozwala to na obiektywną ocenę skuteczności zaproponowanej metody detekcji. Umożliwia to przeprowadzenie analizy jakości i wyznaczenia miar precyzji, co jest kluczowe z punktu widzenia walidacji.

Szczegółowe informacje dotyczące struktury zbioru danych, liczby przepływów, liczby cech oraz rozkładu klas zostały opracowane na podstawie analizy przedstawionej w pracy „A detailed analysis of CICIDS2017 dataset for designing Intrusion Detection Systems” autorstwa R. Panigrahi i S. Borah [25], która stanowi główne źródło statystycznego opisu zbioru oraz jego charakterystyki pod kątem projektowania systemów IDS (ang. Intrusion Detection System). Jednocześnie istotna część informacji dotyczących sposobu generowania danych, architektury środowiska testowego oraz scenariuszy ataków pochodzi z oryginalnej publikacji opisującej proces tworzenia zbioru CICIDS2017, przedstawionej w pracy opublikowanej przez Sharafaldin et al. na konferencji ICISSP (Scitepress, 2018) [24].

7.2 Ataki zawarte w CICIDS2017

Zakładając chęć wykrywania ataków zero-day, należy ponownie zaznaczyć, że w tej pracy termin atak zero-day rozumiany jest jako atak, którego charakterystyka nie jest znana na etapie uczenia modelu, a nie jako konkretna, zidentyfikowana podatność opisana w bazach CVE. To rozumowanie opiera się na fakcie, że każdy atak miał swój debiut jako nowy, nieopisany zero-day. Badania przeprowadzone na znanych atakach (Tabela 7.1) ze względu na dogłębne rozumienie

używanych technik. Przy omówionym wcześniej założeniu, z punktu widzenia modeli, wykryta anomalia nie posiada jednoznacznej etykiety, czym jest ów atak, lecz informuje o pojawieniu się w sieci zachowania odbiegającego od normy z dużym prawdopodobieństwem, że dany przepływ jest atakiem.

Zbiór CIC-IDS2017 zawiera różne klasy ataków oraz realistyczną dystrybucję ilościową [23], [24]:

- **Brute Force:** Atak opierającym się na automatycznym i uporczywym próbom odgadnięcia hasła w celu uzyskania dostępu. Sam atak można porównać do próby otworzenia zamka kombinacyjnego wybierając próbując wszystkie możliwe kombinacje kodu. W środowisku tworzenia bazy testowej CICIDS2017 wykorzystując tą strategię na protokołach FTP i SSH, przy użyciu narzędzia napisanego języku Python, o nazwie "Patator". Duża liczba nieudanych prób uwierzytelnienia w krótkim przedziale czasowym stanowi typową cechę ataków typu brute force. W takich warunkach obserwuje się intensywny, zągęszczony strumień pakietów sieciowych generowanych przez mechanizm wielokrotnych prób dostępu. Jednocześnie nieudane próby logowania nie wiążą się z transmisją plików o dużych rozmiarach, co skutkuje relatywnie niskim zużyciem pasma oraz małą liczbą przesyłanych bajtów [24], [26].

- **FTP-Patator** – Penetracja usługi w którym narzędzie atakowało protokół przesyłu plików (File Transfer Protocol) w celu uzyskania dostępu do przetrzymywanego przez serwer danych.
- **SSH-Patator** – Atak w którym narzędzie atakowało serwer w celu uzyskania dostępu do zdalnego połączenia Secure Shell.

- **DoS / DDoS:**

Rozproszona odmowa usługi (ang. Distributed Denial of Service) jest strategią, której celem nie jest uzyskanie dostępu, lecz przeciążenie serwisu. Celem atakującego jest przeciążenie serwera samą ilością zapytań. Takie zalenie może podochodzić od jednego źródła (DoS), lub być dystrybuowana przez wiele źródeł (DDoS). DDoS wykorzystuje najczęściej wiele wcześniej zainfekowanych urządzeń, co utrudnia filtrację zapytań oraz odróżnienie ataku od normalnego zwiększenia obciążenia serwisu wynikającego z większej liczby zapytań od zwykłych użytkowników. W procesie tworzenia zbioru danych zaznaczono że użyto wielu technik, lecz zaznaczono te bardziej znaczące [23], [24].

- **DoS slowloris** – Dos Slow HTTP Headers mają na celu wykorzystaniu słabości w logice serweta WWW. Nie polega on na zalaniu w kontekście ilości zapytań lecz na przeciążeniu serwera poprzez powolne dosyłanie nagłówków żądania HTTP, czego rezultatem jest sztuczne utrzymywanie kilkuset równoczesnych połączeń z atakowanym serwerem. Takie działanie sprawia, że po krótkim czasie cała „uwaga” serwerów WWW skierowana jest na atakującego, przez co zwiększa nie tylko obciążenie systemu, ale przede wszystkim uniemożliwia innym klientom podłączenie do serwera. Docelowy serwer WWW jest obciążany bardzo dużą liczbą połączeń, przy czym celem ataku jest utrzymanie ich aktywności przez możliwie najdłuższy czas. W trakcie tego typu oddziaływania obserwuje się znaczną liczbę niekompletnych pakietów TCP z ustawioną flagą SYN, charakterystycznych dla inicjowania połączeń. Rozmiary pakietów pozostają niewielkie, co przekłada się na niskie zużycie pasma oraz małą liczbę przesyłanych bajtów [6], [27].

- **DoS Slowhttptest** – Atak z użyciem narzędzia slowhttptest, które implementuje różne warianty ataków Slow HTTP: Slowloris (powolne nagłówki), Slow POST (powolne body), Slow Read (wolne odczytywanie) oraz Range Attack. Każdy tryb wykorzystuje inną metodę, ale wspólnym celem jest wyczerpanie puli dostępnych połączeń serwera przy minimalnym zużyciu przepustowości po stronie atakującego. Atak ten jest relatywnie prosty do przeprowadzenia, ponieważ nie wymaga ani wysokiej przepustowości łącza, ani zaawansowanych zasobów sprzętowych. Jednocześnie jego detekcja w strumieniu ruchu internetowego jest utrudniona, gdyż generowany ruch przypomina legalne żądania HTTP o długim czasie trwania. Istotną przesłanką wskazującą na wystąpienie ataku mogą być jednak połączenia charakteryzujące się rozmiarem okna TCP znacznie mniejszym niż w przypadku normalnego ruchu sieciowego [6], [28].
- **DoS Hulk** – HTTP-Unbearable-Load-King to atak narzędziem napisanym w Python. Zasada działania polega na generowaniu dużej liczby unikalnych żądań HTTP, z których każde wymusza na serwerze wygenerowanie indywidualnej odpowiedzi (poprzez opcję no-cache). Indywidualna odpowiedź nakłada obciążenie obliczeniowe do każdego żądania, a dzięki losowaniu wartości nagłówka każde żądanie wygląda jakby pochodziło z innego źródła. Narzędzie HULK posiada zdolność maskowania rzeczywistego identyfikatora klienta (User-Agent). W trakcie ataku generowane są zarówno zalewowe pakiety TCP-SYN, jak i liczne żądania typu HTTP GET. W celu dokładniejszej analizy mechanizmu działania tego ataku zasadne jest rozpatrzenie osobno metod TCP-SYN flood oraz HTTP-GET flood[6], [29].
- **DoS GoldenEye** – Narzędzie oparte na algorytmie działania HULK, ale idzie krok dalej, symulując liczne aktywne połączenia do serwera z wykorzystaniem żądań Keep-Alive. Przekłada się to na nie tylko zmuszanie do kosztowych obliczeń bez cache, ale i do utrzymania połączenia, co efektywnie zmniejsza ilość zapytań [30].
- **DDoS LOIT** – DDoS Low Orbit Ion Cannon jest narzędziem stworzone przez Praetox Technology, do stress-testowania aplikacji sieciowych. Sama pop kulturowa nazwa może sugerować zachowanie narzędzia. Zasada działania polega na dokładnej definicji DDoS przedstawionej wcześniej. Narzędzie zalewa cel pakietami TCP, UDP i HTTP. Jego zadaniem jest symulacja ekstremalnego popytu na usługi aplikacji i ogromnego ruchu [31].

- **Web Attack:**

Ataki Webowe atakują bezpośrednio logikę serwisu utrzymywanego na danym serwerze. Ich celem jest eksploatacja błędów w implementacji usługi poprzez interakcję z samym interfejsem dostępnym dla normalnego użytkownika. Nie atakują bezpośrednio infrastruktury jako instancji tylko wykorzystują prawidłowo działające funkcjonalności aplikacji w sposób niezamierzony przez twórców [24].

- **Brute Force** – Tak jak w przypadku Brute Force w FTP i SSH, przy użyciu narzędzia Patator, atak ten polega na metodycznym algorytmie prób i błędów. Przykładem Web Brute Force byłoby iteracyjne zgadywanie kluczy do API [24].
- **XSS** – Cross-Site Scripting, jest techniką zastrzykującą (ang. injection) złośliwy kod w wrażliwe bądź niezabezpieczone miejsce, Złośliwa zawartość wysyłana może być przez przeglądarkę internetowej. Często przyjmuje formę segmentu kodu JavaScript, ale może również obejmować kod HTML, Flash lub dowolny inny typ kodu, który przeglądarka może wykonać [32].

- **Sql Injection** – Atak na aplikacje webowe, który wykorzystuje niezrewalidowane dane wejściowe w zapytaniach SQL. Podobnie jak XSS, wykorzystuje niezabezpieczone bądź podatne fragmenty logiki aplikacji internetowej. SQL Injection jest typowo skupione na logice komunikacji z bazą danych. Atak wykorzystuje podatne zapytania SQL zmieniające na takie, które mają na celu na przykład wyświetlić większą ilość danych niż powinny, bądź co groźniejsze edytować danych dane na serwerze.

- **Heartbleed Attack:**

Heartbleed to poważna podatność w popularnej bibliotece kryptograficznej OpenSSL. Błąd ten umożliwia nieautoryzowany dostęp do pamięci systemów chronionych przez podatne wersje OpenSSL. W rezultacie możliwe jest podsłuchiwanie połączeń, kradzież danych oraz podszywanie się pod legitymowane usługi i użytkowników. Atak Heartbleed został przeprowadzony przy użyciu narzędzia Heartleech. W scenariuszu testowym zainstalowano celowo podatną wersję OpenSSL 1.0.1f, a następnie za pomocą Heartleech dokonano zrzutów pamięci procesu serwera WWW. W czasie trwania ataku widoczne są wyraźne odchylenia od standardowego wzorca ruchu sieciowego. Pakiety przychodzące mają długość mniejszą niż minimalny rozmiar komunikatu heartbeat, który wynosi 20 bajtów. Natomiast pakiety wysyłane przez system osiągają rozmiary rzędu kilobajtów, co w kontekście odpowiedzi typu heartbeat jest nietypowe i wskazuje na nieprawidłowe działanie protokołu.

Istotnym sygnałem umożliwiającym rozpoznanie ataku jest znacząca asymetria pomiędzy wielkością danych odbieranych i nadawanych. W normalnych warunkach operacyjnych mechanizm heartbeat generuje komunikaty o jednakowej długości w obu kierunkach transmisji. W przypadku ataku obserwuje się jednak sytuację, w której wiadomości inicjujące są nienaturalnie krótkie, natomiast odpowiedzi sieciowe mają nadmiernie duży rozmiar [24], [33].

- **Infiltration:**

Jest to ogólna nazwa na ataki mając oznaczyć włamanie się do systemu za pomocą Stachnik socjalnych i nie tylko. Od wewnątrz zazwyczaj polega na wykorzystaniu podatnego oprogramowania, takiego jak Adobe Acrobat Reader. Po udanym ataku, na komputerze ofiary zostanie uruchomiona tylna furka (ang. back door), która może przeprowadzić różne ataki [24].

- **Botnet ARES:**

Istnieje wiele narzędzi służących do przeprowadzania ataków botnetowych, takich jak Grum, Windigo, Storm oraz Ares. W zbiorze danych CICIDS2017 wykorzystano narzędzie Ares Botnet napisany w języku Python, umożliwiający zdalne sterowanie powłoką systemową (remote shell), przesyłanie i pobieranie plików, wykonywanie zrzutów ekranu oraz rejestrowanie naciśnięć klawiszy (ang. keylogging) [24].

- **Port Scan:**

Jest to technika triasu przed atakiem. Polega, jak sama nazwa wskazuje, na skanowaniu portów w poszukiwaniu otwartych. Atakujący wysyła wiadomość na dany port i z uzyskanej odpowiedzi uzyskuje informacji czy port jest w użyciu, ale nie tylko. Na podstawie odpowiedzi atakujący jest w stanie dowiedzieć się czy jedna ze znanych podatności protokołu jest zawarta w skanowanym porcie [34].

TABELA 7.1: Dystrybucja ilościowa przepływów oznaczonych i przypisanych do poszczególnych ataków

Nazwa	Ilość
Benign	2359289
Faulty	288602
DoS Hulk	231073
PortScan	158930
DDoS	41835
DoS GoldenEye	10293
FTP-Patator	7938
SSH-Patator	5897
DoS slowloris	5796
DoS Slowhttptest	5499
Bot	1966
Web Attack – Brute Force	1507
Web Attack – XSS	652
Infiltration	36
Web Attack – SQL Injection	21
Heartbleed	11

7.3 Charakterystyka zbioru danych CICIDS2017

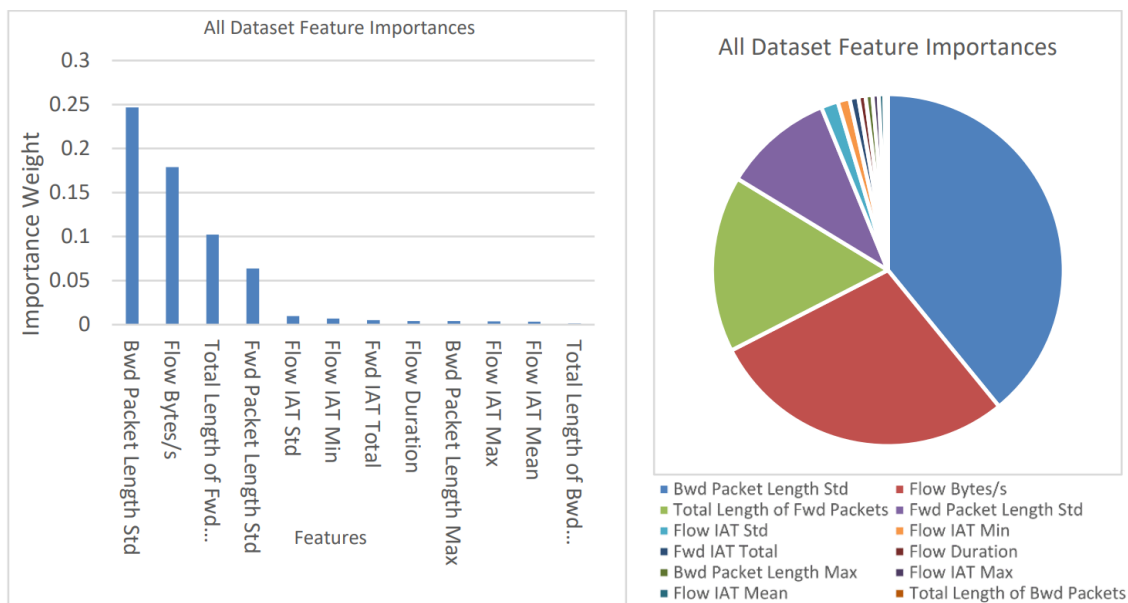
Omawiany zbiór danych został zaprojektowany w strukturze pięciu dni tygodnia, od poniedziałku do piątku, opisując przeprowadzone w tych dniach testy ataków. Wyjątek stanowi poniedziałek, w którym zawarto tylko i wyłącznie normalny ruch sieciowy. Po połączeniu danych w jeden zbiór oraz usunięciu błędnych bądź niekompletnych rekordów, zbiór zawiera 2 830 540 przepływów sieciowych. Każdy z przepływów posiada 83 cechy oraz etykietę klasy ataku. Cechy to między innymi czas trwania przepływu, wielkość średnia pakietów, a także charakterystyki czasowe. Zbiór obejmuje 14 wcześniej opisanych w podrozdziale „Ataki zawarte w CICIDS2017” ataków, oraz ruch normalny którego etykieta nosi nazwę „BENIGN” (pl. łagodny). Ruch normalny stanowi znaczną większość obserwacji, natomiast niektóre z ataków występują rzadziej niż sporadycznie. Taka niezbilansowana struktura nie jest przypadkowa. Ma ona na celu odzwierciedlenie naturalnego ruchu sieciowego. Zbiór zawiera również ataki, które nie charakteryzują się wysoką intensywnością ruchu sieciowego, nie generują dużej liczby przepływów i tym samym pozostawiają ograniczoną ilość śladów w danych sieciowych. Przykładem takiego scenariusza jest atak Heartbleed, który występuje relatywnie rzadko i nie wymaga inicjowania wielu połączeń. W praktyce napastnik jest w stanie pozyskać poufne informacje z obszaru pamięci serwera poprzez wysłanie jedynie kilku celowo skonstruowanych pakietów typu Heartbeat (Tabela 7.1), [24], [25].

7.4 Przygotowanie i przetwarzanie danych

Do przeprowadzenia badania, czy modele uczenia maszynowego nienadzorowanego mają potencjał wykrywania nieznanymi ataków tylko dzięki treningowi na normalnych danych, konieczne było przygotowanie danych. Mimo że oczekiwanym rezultatem jest przedstawienie wyniku, który mówi że modele są w stanie odróżnić sygnatury ataków na podstawie różnych parametrów, ustalono że nie wszystkie cechy przepływów powinny być brane pod uwagę. Wybór ten został dokonany na podstawie analizy istotności cech przedstawionej w pracy Kahramana Kostasa z Uniwersytetu w Essex, pod tytułem „Anomaly Detection in Networks Using Machine Learning” [6]. Autor demonstruje sposób, dzięki któremu ilość cech można ograniczyć z 83 do 20. W pierwszej kolejności w cytowanej pracy podkreślono, że eliminacja cech o charakterze typowo identyfikacyjnych, takich jak adresy IP, numery portów czy znaczniki czasowe, może prowadzić nadmierne dopasowanie modeli do konkretnego środowiska sieciowego, co może się przekładać na nadmierną ilość błędnych detekcji spowodowanych zwykłym ruchem. Zarekomendowano pozbycie się tych cech i skupienie się na cechach opisujących „kształt” ruchu sieciowego. Dodatkowo, dzięki temu, niewielkie zmiany konfi-

guracyjne i eksploatacja sieci nie będą problematyczne. Po usunięciu tych cech autor zwraca się do uczenia maszynowego, aby zdeterminować przydatność pozostałych cech. Zastosowano algorytm Random Forest Regressor do wyznaczenia wag istotności cech. Najwyższe wartości osiągnęły cechy oparte na statystykach czasowych i rozmiarowych takie jak Flow Duration, Flow Bytes/s, Bwd/Fwd Packet Length (mean, max, std) oraz liczba i łączna długość pakietów w obu kierunkach (rys. 7.2). Te cechy zostały użyte do treningu modeli nienadzorowanych ML w tym badaniu.

Cytując tą pracę należy wspomnieć o fakcie że w niej poruszone zostało bardzo podobne pytanie jednak kluczową różnicą jest podejście. Podczas gdy w tej pracy główną myślą jest szkolenie modeli uczenia nienadzorowanego Kahrman Kotasa skupia się na uczeniu nadzorowanym z wykorzystaniem etykiet ataków. Mimo tego jego statystyczne podejście i podobny cel sprawiło że tematyka rozdziału 3.2 [6]. została wykorzystana w tej pracy do wyboru cech.



RYSUNEK 7.1: Statystyki pochodzące z badania ważności cech ilustrowane na wykresach Źródło: K. Kostas, Anomaly Detection in Networks Using Machine Learning, 2021 [6]

TABELA 7.2: Reprezentacja wyników badania istotności cech dla zbioru CIC-IDS2017. Źródło: K. Kostas, Anomaly Detection in Networks Using Machine Learning, 2021 [6]

Nazwa Cechy	Współczynnik Wartości	Nazwa Cechy	Współczynnik Wartości
Bwd Packet Length Std	0.246627	Flow IAT Mean	0.003266
Flow Bytes/s	0.178777	Total Length of Bwd Packets	0.001305
Total Length of Fwd Packets	0.102417	Fwd Packet Length Min	0.000670
Fwd Packet Length Std	0.063889	Bwd Packet Length Mean	0.000582
Flow IAT Std	0.009898	Flow Packets/s	0.000541
Flow IAT Min	0.006946	Fwd Packet Length Mean	0.000526
Fwd IAT Total	0.005121	Total Backward Packets	0.000169
Flow Duration	0.004150	Total Fwd Packets	0.000138
Bwd Packet Length Max	0.004007	Fwd Packet Length Max	0.000125
Flow IAT Max	0.003579	Bwd Packet Length Min	0.000084

TABELA 7.3: Intuicyjne wyjaśnienie cech branych pod uwagę w trenowaniu i testowaniu modeli [6]

Nazwa Cechy	Wyjaśnienie znaczenia
Bwd Packet Length Std	Odchylenie standardowe rozmiarów pakietów serwera. Im większe, tym większe wahania w rozmiarze pakietów. Wysokie wartości mogą wskazywać np. na mieszany ruch (kontrolny i danych).
Flow IAT Mean	Średni czas między przepływami; długie przerwy mogą wskazywać „zamrożone” połączenie lub oczekiwanie klienta na serwer. Przydatne do wykrywania timeoutów i opóźnień.
Flow Bytes/s	Średnia liczba bajtów przesyłanych na sekundę w całym flowie. Użyteczne do analizy przepustowości i intensywności ruchu. Wysokie wartości = szybka transmisja.
Total Length of Bwd Packets	Ile bajtów wysłał serwer w odpowiedzi (backward). Suma wszystkich danych wysłanych w tym flowie. W parze z forwardem wskazuje, kto był bardziej aktywny.
Total Length of Fwd Packets	Ile bajtów wysłał klient (forward). Może wskazywać np. najmniejszy pakiet sterujący (ACK) bez danych. Suma wszystkich bajtów wysłanych od klienta w flowie
Fwd Packet Length Min	Najmniejszy pakiet klienta. Może być sygnałem sterującym, np. potwierdzenie (ACK), które nie zawiera danych.
Fwd Packet Length Std	Odchylenie standardowe rozmiarów pakietów klienta. Niskie wartości – pakiety podobnej wielkości; wysokie wartości – duża zmienność, np. nieregularny ruch lub malware.
Bwd Packet Length Mean	Średnia wielkość pakietów wysyłanych przez serwer. Pomaga określić, jak „ciężkie” są dane wracające z serwera. Ma znaczenie przy analizie wydajności i typów ruchu.
Flow IAT Std	Odchylenie czasu między pakietami w całym flowie. Pomaga rozpoznać nieregularność – np. malware może mieć losowy rytm pakietów.
Flow Packets/s	Średnia liczba pakietów na sekundę. Pomaga zrozumieć tempo „rozmowy” sieciowej. Mała ilość danych + dużo pakietów = chat, VoIP lub malware.
Flow IAT Min	Najkrótsza przerwa między pakietami. Może pokazywać, jak szybko dane były przesyłane – najniższy możliwy „takt”.
Fwd Packet Length Mean	Średni rozmiar pakietów klienta. Uśrednia wszystkie pakiety forward. Pomaga klasyfikować typ ruchu (plik, rozmowa, ping itd.).
Fwd IAT Total	Całkowity czas (suma) między pakietami klienta. Duża wartość = pakiety wysyłane w dużych odstępach; mała wartość = intensywna, szybka komunikacja.
Total Backward Packets	Ile pakietów wysłał serwer z powrotem do klienta (backward). Zestawione z forward pokazuje dominację danych.
Flow Duration	Całkowity czas trwania rozmowy sieciowej (w mikrosekundach). Dłuższy czas = dłuższe połączenie.
Total Fwd Packets	Ile pakietów wysłał klient do serwera (forward). Może wskazywać na wielkość wysyłanych danych.
Bwd Packet Length Max	Największy pakiet wysłany przez serwer. Pokazuje maksymalną jednostkę danych w jednym pakiecie. Może sugerować typ odpowiedzi (plik, HTML, wideo).
Fwd Packet Length Max	Największy pojedynczy pakiet wysłany przez klienta. Pokazuje maksymalną jednostkę danych; duże wartości = przesył pliku lub wideo.
Flow IAT Max	Najdłuższa przerwa między pakietami. Może wskazywać „zamrożenie” połączenia lub oczekiwanie klienta.
Bwd Packet Length Min	Minimalny rozmiar pakietów serwera. Pomaga wykryć wahania w przesyłanych danych.
Repetition_num	Dodatkowa cecha dodana po usunięciu powtarzających się przepływów.

7.5 Proces przetwarzania danych

Dane z poniedziałku posiadają tylko normalny ruch, posłużyły do treningu modeli, natomiast dane z reszty dni zostały scalone i wykorzystane do przeprowadzenia testów skuteczności modeli. Proces przygotowania danych testowych i treningowych został przeprowadzony etapowo z wykorzystaniem bibliotek „Pandas”, „NumPy” oraz „Matplotlib”. Operacje przetwarzania realizowano na danych zapisanych w plikach w formacie „csv”, które następnie zostały zaimportowane i przekształcone do struktur typu „DataFrame” biblioteki „Pandas”. Tak przygotowane dane stanowiły podstawę do dalszych etapów analizy oraz budowy modeli uczenia maszynowego:

1. Usunięcie białych znaków z nazw kolumn.

W celu zachowania spójności struktury danych oraz uniknięcia potencjalnych błędów podczas dalszego przetwarzania, usunięto białe znaki z nazw kolumn. Pliki CSV często zawierają niezamierzone spacje w nagłówkach, co może prowadzić do problemów przy odwoływaniu się do cech w kodzie.

2. Usunięcie cechy *Fwd Header Length.1*.

Cecha ta jest duplikatem kolumny *Fwd Header Length* i nie wnosi dodatkowej informacji opisowej. Jej obecność mogłaby wprowadzać redundancję i niepotrzebnie zwiększać wymiarowość danych, dlatego została usunięta [6].

3. Selekcja cech istotnych.

Usunięto cechy uznane za mało istotne, pozostawiając wyłącznie atrybuty wskazane jako najbardziej znaczące w literaturze (Tabela 7.2). Zabieg ten pozwala ograniczyć wpływ szumu oraz poprawić zdolność modeli do generalizacji [6].

4. Usunięcie przepływów z błędnymi lub niepełnymi wartościami.

Nie wszystkie przepływy sieciowe zostały poprawnie zarejestrowane. Część z nich zawierała wartości nieliczbowe (NaN) lub nieskończone (Inf), które uniemożliwiają poprawne działanie algorytmów uczenia maszynowego. Takie rekordy zostały usunięte w celu zapewnienia jakości danych wejściowych.

5. Usunięcie powtarzających się przepływów.

W celu uniknięcia stroniczości modeli oraz sztucznego zawyżenia gęstości wybranych obserwacji, ze zbioru danych usunięto powtarzające się przepływy. Duplikaty nie wnoszą dodatkowej informacji opisowej na poziomie cech. Jednocześnie sam fakt wielokrotnego występowania danego przepływu może mieć wartość informacyjną, dlatego podczas przetwarzania danych wprowadzono dodatkową kolumnę „Repetition_num”, przechowującą liczbę powtórzeń danego rekordu.

6. Usunięcie kolumny „Label”.

Na końcowym etapie przygotowania danych usunięto kolumnę zawierającą etykiety ataków. Operacja ta została wykonana celowo na samym końcu, aby wcześniej zapisać etykiety w osobnej strukturze danych (df_label). Umożliwia to późniejsze porównanie wyników predykcji modeli z rzeczywistymi oznaczeniami i przeprowadzenie analizy statystycznej skuteczności detekcji.

7. Skalowanie danych.

Skalowanie danych polega na sprowadzeniu wszystkich cech numerycznych do porównywalnej skali, tak aby żadna z nich nie dominowała pozostałych wyłącznie ze względu na rząd wielkości. W tym celu zastosowano metodę standaryzacji zaimplementowaną w klasie „StandardScaler” z biblioteki „scikit-learn”.

Skalowanie jest kluczowe dla każdego z zastosowanych modeli:

- *Isolation Forest* — Algorytm jest wrażliwy na samą skalę cech. Nie bazuje bezpośrednio na odległościach, skala cech wpływa na sposób losowego podziału przestrzeni cech, co może zaburzać proces izolowania obserwacji.
- *Local Outlier Factor* — Model opiera się na obliczaniu odległości między punktami danych. Brak skalowania prowadzi do dominacji cech o większych wartościach numerycznych i zniekształcenia lokalnej gęstości.
- *Autoencoder* — Sieci neuronowe są wrażliwe na skalę danych wejściowych. Brak standaryzacji utrudnia proces optymalizacji, spowalnia działanie modelu to niakceptowalnych wartości.

Po zakończeniu przygotowania testowa baza danych została zredukowana do 20 cech, 1 676 993 przepływów dla danych testowych i 400 676 dla danych treningowych.

7.6 Środowisko badawcze

Do realizacji badań przygotowano odpowiednie środowisko obliczeniowe. Eksperymenty przeprowadzono na serwerze Fujitsu RX2520, wyposażonym w 192 GB pamięci RAM oraz 48 wątków obliczeniowych, opartych na dwóch procesorach Intel Xeon E5-2420 v2 (6 rdzeni, 2.20 GHz).

Na serwerze uruchomiono maszynę wirtualną, na której skonfigurowano środowisko *Jupyter Notebook*. Maszynie wirtualnej przydzielono 128 GB pamięci RAM oraz połowę dostępnych zasobów procesora. Takie środowisko umożliwiło efektywne przetwarzanie dużych zbiorów danych oraz trenowanie modeli uczenia maszynowego.

Kod źródłowy przeprowadzonych badań jest ogólnodostępny i został udostępniony przez autora na platformie GitHub pod adresem: https://github.com/m-icha-l/Anomaly_detection_ML

W implementacji części programowej wykorzystano następujące technologie i biblioteki:

- **Python 3.11** — Główny język programowania użyty do implementacji całego procesu badawczego, w tym przetwarzania danych, budowy modeli oraz analizy wyników.
- **scikit-learn** — Biblioteka wykorzystywana do implementacji algorytmów uczenia maszynowego, w szczególności modeli detekcji anomalii takich jak *Isolation Forest* oraz *Local Outlier Factor*, a także do skalowania danych i obliczania metryk ewaluacyjnych.
- **PyTorch** — Framework użyty do implementacji Autoenkodera, umożliwiający definiowanie architektury sieci neuronowej, przeprowadzanie procesu uczenia oraz obliczanie błędu rekonstrukcji.
- **Pandas** — Biblioteka służąca do wczytywania danych z plików CSV, ich czyszczenia, selekcji cech, agregacji oraz przygotowania zbiorów treningowych i testowych w postaci ramek danych.
- **NumPy** — Wykorzystywana do operacji numerycznych, transformacji danych oraz efektywnego przetwarzania macierzy i wektorów numerycznych.

- **Matplotlib** — Biblioteka używana do wizualizacji danych oraz wyników eksperymentów, w tym wykresów rozkładów cech, histogramów błędów rekonstrukcji oraz prezentacji wyników detekcji anomalii.
- **Seaborn** — Narzędzie wspomagające tworzenie bardziej czytelnych i estetycznych wizualizacji statystycznych, takich jak macierze korelacji.
- **logging** — Moduł wykorzystany do rejestrowania przebiegu eksperymentów, parametrów modeli oraz informacji diagnostycznych, co ułatwia analizę i powtarzalność badań.

Rozdział 8

Badania i analiza wyników

8.1 Metody oceny wydajności

Podczas obliczeń wydajności każda z predykcji mogła mieć wynik będący jednym z 4 klasyfikacji:

- **TP** – True Positive (pl. Prawdziwie Pozytywny) – Opisuje on wynik, w którym model uznał dany przepływ za anomalię i po weryfikacji okazał się on rzeczywiście częścią ataku.
- **TN** – True Negative (pl. Prawdziwie Negatywny) – Opisuje on wynik, w którym model uznał dany przepływ za część normalnego ruchu sieciowego i po weryfikacji okazał się on rzeczywiście częścią normalnego ruchu.
- **FP** – False Positive (pl. Fałszywie Pozytywny) – Opisuje on wynik, w którym model uznał dany przepływ za anomalię, a po weryfikacji okazał się on częścią normalnego ruchu sieciowego, model się pomylił.
- **FN** – False Negative (pl. Fałszywie Negatywny) – Opisuje on wynik, w którym model uznał dany przepływ za część normalnego ruchu sieciowego, a po weryfikacji okazało się jest on częścią ataku, model się pomylił.

Do oceny wydajności wykorzystano klasyczne podejście do modeli uczenia maszynowego. Sposób oceny wydajności i parametry zostały zaimplementowane według tych samych metod co w pracy „Anomaly Detection in Networks-Using-Machine Learning” [6] i były to:

- Dokładność (ang. Accuracy):

Miara określająca udział wszystkich poprawnych predykcji modelu w całkowitej liczbie obserwacji, obliczana według wzoru [35]:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (8.1)$$

- Czulość (ang. Sensitivity, Recall):

Miara określająca zdolność modelu do poprawnego wykrywania przypadków pozytywnych (np. ataków), dana wzorem [35]:

$$Sensitivity = Recall = \frac{TP}{TP + FN} \quad (8.2)$$

- Precyzja (ang. Precision):

Miara określająca, jaka część obserwacji zaklasyfikowanych jako pozytywne jest faktycznie pozytywna [35]:

$$Precision = \frac{TP}{TP + FP} \quad (8.3)$$

- F-measure (F1-score):

Miara będąca średnią harmoniczną precyzji oraz czułości, łącząca obie te własności w jedną wartość [35]:

$$F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (8.4)$$

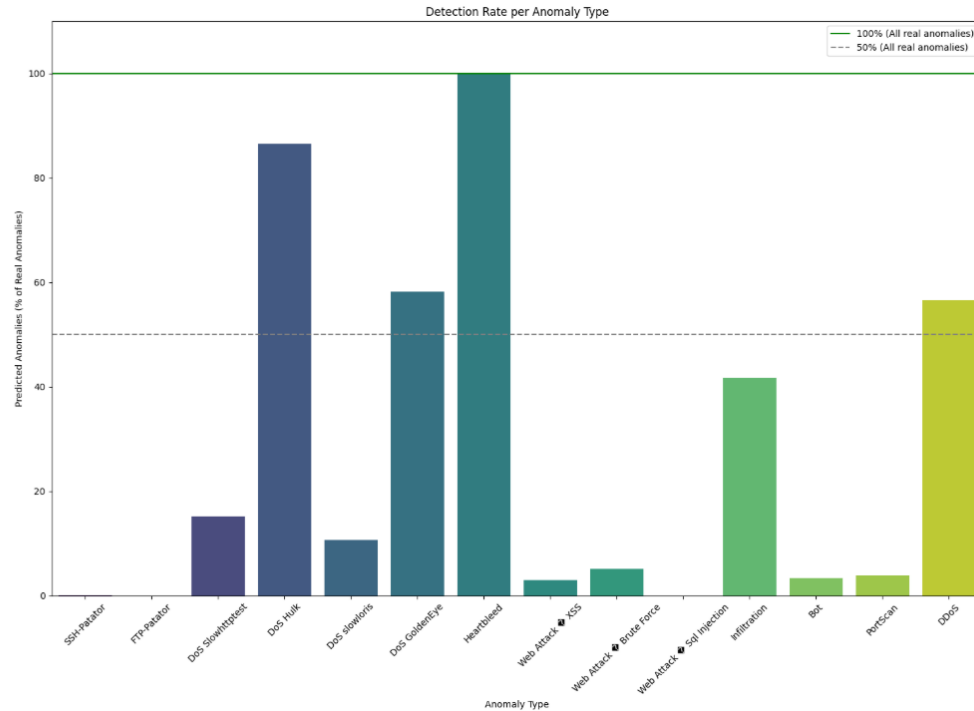
8.2 Wyniki model Isolation Forest

Model Isolation Forest został przygotowany do testów jako pierwszy. Wyniki oceny modelu przedstawiono w tabeli 8.1 Szczegółową analizę skuteczności wykrywania poszczególnych typów ataków zaprezentowano w tabeli 8.2, jaki i zostały wyświetlone przez program w postaci wykres słupkowego dla wygodniejszej wizualizacji (rys. 8.1).

IF osiągnął bardzo zadowalający wynik dokładności. Poziom 91% jest dobrym wynikiem i sugeruje, że ilość poprawnych predykcji znacznie przewyższa ilość błędnych, w kontekście poprawnie odnalezionych anomalii jak i poprawnie rejestrowanie zwykłego ruchu sieciowego. Wartość precyzji wynosząca 86,66% wskazuje na niski odsetek fałszywych alarmów (FP). Niepokojącą wartością jest natomiast czułość na poziomie tylko 68,20%. Nie jest to zadowalający wyniki, ponieważ można go sparafrazować, że około jedna trzecia rzeczywistych ataków pozostaje niewykryta. Wartość średniej harmonicznej wynosi 76,33%, sugeruje umiarkowany balans precyzji i czułości. Szczegółowa analiza skuteczności wykrywania poszczególnych typów ataków (tabela 8.2) oraz jej wizualizacja przez program (rys. 8.1) ujawnia znaczące różnice w wydajności modelu. Najlepsze wyniki osiągnięto dla ataku Heartbleed (100% poprawnych detekcji) oraz DoS Hulk (86,47%). Z drugiej strony, model wykazał bardzo niską skuteczność w identyfikacji ataków SSH-Patator (0,06%), FTP-Patator (0%) oraz Web Attack SQL Injection (0%), co wskazuje na trudności w wykrywaniu ataków opartych na brute-force oraz ataków aplikacyjnych. Wyniki te sugerują, że model Isolation Forest jest szczególnie efektywny w wykrywaniu ataków powodujących anomalie w volumenie ruchu sieciowego (DoS/DDoS), natomiast wymaga uzupełnienia w kontekście ataków charakteryzujących się subtelniejszymi wzorcami.

TABELA 8.1: Wyniki miar jakości modelu Isolation Forest

Miara	Wartość
Accuracy	91,54%
Sensitivity	68,20%
Precision	86,66%
F-measure	76,33%



RYSUNEK 8.1: Wykres ilustrujący wyniki modelu Isolation Forest w ramach Pipeline

TABELA 8.2: Skuteczność detekcji poszczególnych typów ataków przez model Isolation Forest

Typ ataku	Poprawnie wykryte [%]
SSH-Patator	0,0634
FTP-Patator	0
DoS Slowhttptest	15,2286
DoS Hulk	86,4726
DoS slowloris	10,5909
DoS GoldenEye	58,1558
Heartbleed	100
Web Attack XSS	2,9141
Web Attack Brute Force	5,1156
Web Attack SQL Injection	0
Infiltration	41,6667
Bot	3,2909
PortScan	3,8482
DDoS	56,5677

8.3 Wyniki modelu Local Outlier Factor

Kolejnym przetestowanym algorytmem był model Local Outlier Factor (LOF). Wyniki oceny modelu przedstawiono w tabeli 8.3, natomiast szczegółową analizę skuteczności wykrywania poszczególnych typów ataków zaprezentowano w tabeli 8.4.

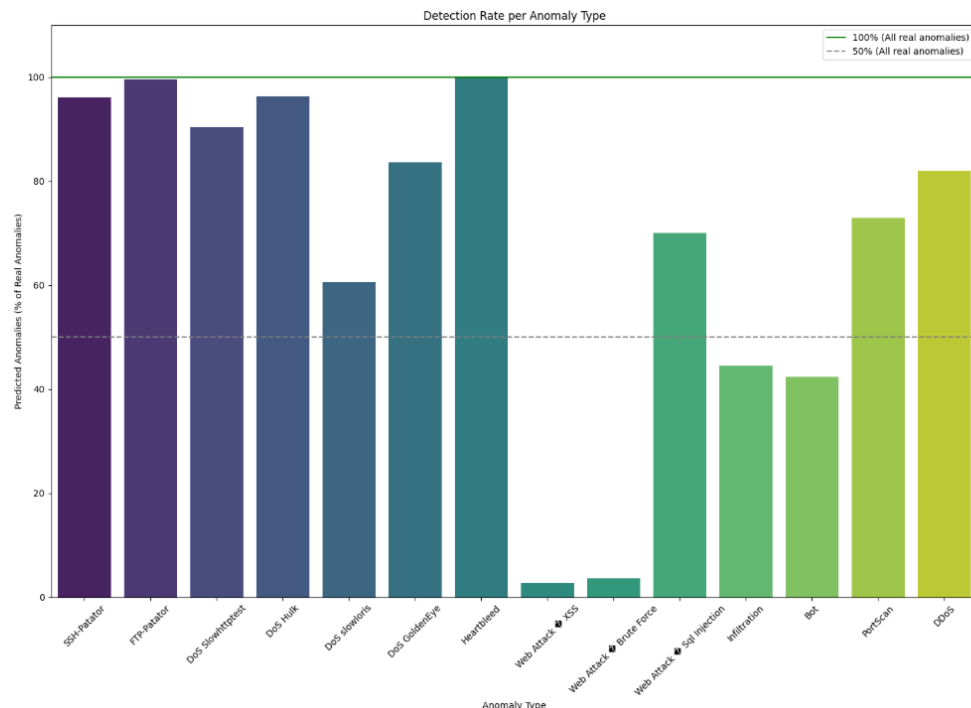
Wyniki modelu LOF znacznie się różnią od tych analizowanych poprzednio. Jako pierwsze zauważono znaczne pogorszenie dokładności (79,92%) oznacza to, że około 30% predykcji było błędne. Znacznie lepszy wynik natomiast otrzymała miara czułości (88,91%) model LOF wykrył

więcej przepływów będących częścią ataku. Niepokojącym wynikiem okazała się znacznie niższa precyzja (49,89%), wskazująca na wysoki odsetek fałszywych alarmów. niemal połowa oznaczonych przez LOF flow ataków okazała się błędna. Wartość F-miary wynosząca 63,91% jest niższa niż w przypadku Isolation Forest, co sugeruje gorszy ogólny balans między precyzją a czułością.

Analiza skuteczności wykrywania poszczególnych typów ataków (tabela 8.4) ujawnia zdecydowanie lepszą wydajność modelu LOF w kontekście ataków typu brute-force oraz DoS. Model osiągnął niemal perfekcyjne wyniki dla ataków FTP-Patator (99,60%), SSH-Patator (96,04%) oraz Heartbleed (100%), co stanowi znaczną poprawę w porównaniu do Isolation Forest. Równie wysoką skuteczność zaobserwowano dla większości wariantów ataków DoS (DoS Hulk: 96,28%, DoS Slowhttptest: 90,43%). Niemniej jednak, model LOF nadal wykazuje znaczące trudności w wykrywaniu ataków aplikacyjnych. Web Attack XSS (2,76%) oraz Web Attack Brute Force (3,57%) pozostają praktycznie niewykrywalne. Interesującą obserwacją jest znaczna poprawa w detekcji Web Attack SQL Injection (70% w porównaniu do 0% w Isolation Forest) co sugeruje, że model LOF lepiej radzi sobie z niektórymi wzorcami ataków aplikacyjnych. Niska skuteczność wykrywania botów (42,33%) oraz ataków typu Infiltration (44,44%) wskazuje na trudności w identyfikacji ataków charakteryzujących się długotrwałym, subtelnym zachowaniem.

TABELA 8.3: Wyniki miar jakości modelu Local Outlier Factor

Miara	Wartość
Accuracy	79,92%
Sensitivity	88,91%
Precision	49,89%
F-measure	63,91%



RYSUNEK 8.2: Wykres ilustrujący wyniki modelu Local Outlier Factor w ramach Pipeline

TABELA 8.4: Skuteczność detekcji poszczególnych typów ataków przez model Local Outlier Factor

Typ ataku	Poprawnie wykryte [%]
SSH-Patator	96,0355
FTP-Patator	99,5953
DoS Slowhttptest	90,4343
DoS Hulk	96,2780
DoS slowloris	60,5537
DoS GoldenEye	83,5716
Heartbleed	100,0000
Web Attack XSS	2,7607
Web Attack Brute Force	3,5739
Web Attack SQL Injection	70,0000
Infiltration	44,4444
Bot	42,3336
PortScan	72,9589
DDoS	82,0237

8.4 Wyniki modelu Autoencoder

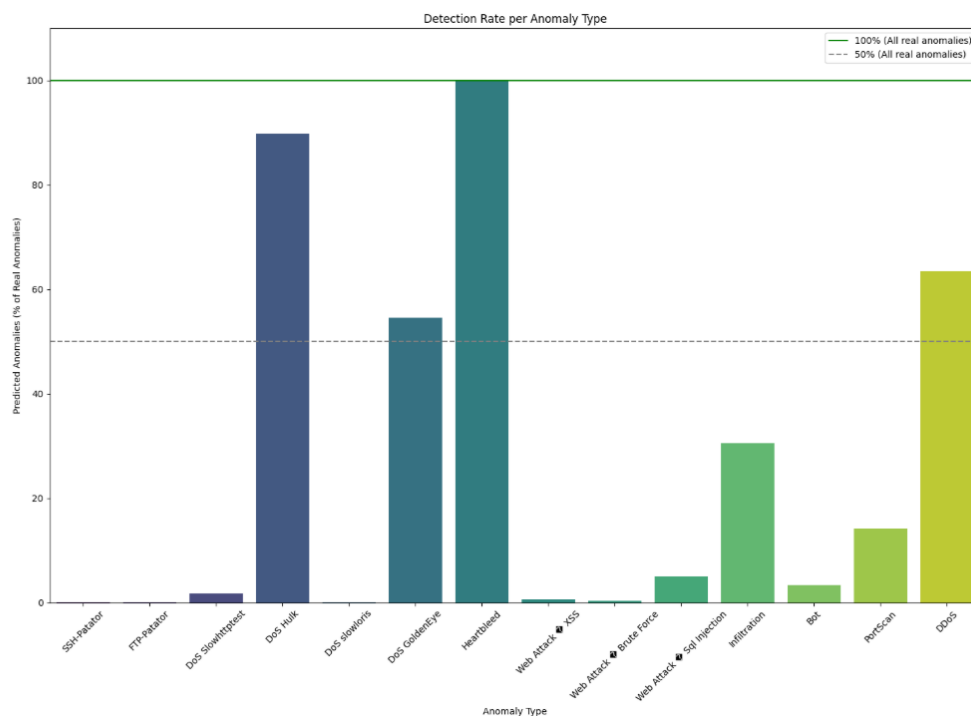
Ostatnim testowanym modelem był Autoencoder. Wyniki oceny modelu przedstawiono w tabeli 8.5, natomiast szczegółową analizę skuteczności wykrywania poszczególnych typów ataków zaprezentowano w tabeli 8.6, a wizualizację danych wyjściowych na rysunku (rys. 8.3).

Model Autoencoder osiągnął najlepsze ogólne wyniki spośród wszystkich testowanych algorytmów. Dokładność na poziomie 91,84% jest porównywalna z Isolation Forest, jednak model wykazał znacznie lepszy balans między czułością (72,09%) a precyzją (84,81%). Wartość F-miary wynosząca 77,93% jest najwyższa spośród wszystkich modeli, co potwierdza optymalne wyważenie między minimalizacją fałszywych alarmów a maksymalizacją wykrywalności rzeczywistych ataków.

Analiza skuteczności wykrywania poszczególnych typów ataków (tabela 8.6) ujawnia interesujący profil wydajności. Model Autoencoder wykazał doskonałe wyniki dla ataków Heartbleed (100%) oraz bardzo wysoką skuteczność dla DoS Hulk (89,79%) i DDoS (63,51%). Podobnie jak Isolation Forest, model radzi sobie dobrze z atakami powodującymi znaczące zmiany w wolumenie ruchu sieciowego. Jednak model Autoencoder odziedziczył również słabości charakterystyczne dla Isolation Forest. Praktycznie zerowa skuteczność w wykrywaniu ataków SSH-Patator (0,06%), FTP-Patator (0,03%) oraz bardzo niska dla DoS slowloris (0,02%) wskazuje na trudności w identyfikacji ataków typu Brute-force i niektórych wariantów DoS. Podobnie niska wydajność dla ataków aplikacyjnych Web Attack XSS (0,61%), Web Attack Brute Force (0,28%) oraz Web Attack SQL Injection (5%) sugeruje, że model ma problemy z wykrywaniem subtelnych anomalii w ruchu aplikacyjnym. Warto zauważyć, że model Autoencoder osiągnął umiarkowaną skuteczność w wykrywaniu ataków Infiltration (30,56%), co jest lepszym wynikiem niż w przypadku Isolation Forest (3,29%), ale znacznie gorszym niż LOF (44,44%). Podobna sytuacja dotyczy skanowania portów (PortScan: 14,20%), gdzie model wykazał wyższą skuteczność niż Isolation Forest, ale niższą niż LOF.

TABELA 8.5: Wyniki miar jakości modelu Autoencoder

Miara	Wartość
Accuracy	91,84%
Sensitivity	72,09%
Precision	84,81%
F-measure	77,93%



RYSUNEK 8.3: Wykres ilustrujący wyniki modelu Autoencoder w ramach Pipeline

TABELA 8.6: Skuteczność detekcji poszczególnych typów ataków przez model Autoencoder

Typ ataku	Poprawnie wykryte [%]
SSH-Patator	0,0634
FTP-Patator	0,0337
DoS Slowhttptest	1,7410
DoS Hulk	89,7864
DoS slowloris	0,0186
DoS GoldenEye	54,5764
Heartbleed	100,0000
Web Attack XSS	0,6135
Web Attack Brute Force	0,2803
Web Attack SQL Injection	5,0000
Infiltration	30,5556
Bot	3,2909
PortScan	14,1966
DDoS	63,5139

Rozdział 9

Wnioski

9.1 Podsumowanie badań

Badania trzech algorytmów wykrywania anomalii w ruchu sieciowym, dały możliwość ocenę potencjału uczenia nienadzorowanego w cyberbezpieczeństwie w sieciach. Isolation Forest, Local Outlier Factor oraz Autoencoder wykazały różne sygnatury charakterystyk i skuteczności (tabela 9.1). Analiza wyników miar wskazuje, że te różnice przekładają się na odmienne właściwości detekcyjne modeli. IF i Autoencoder charakteryzują się relatywnie wysoką precyzją, jednak czułość bywa problematyczna. Skutkiem tego było omijanie części ataków. LOF z drugiej strony osiągał bardzo wysoką czułość, co pozwoliło mu wykryć znaczną większość złośliwych przepływów, jednak kosztem tego była niezadowalająco duża ilość FP.

Analiza skuteczności wykrywania poszczególnych ataków wskazuje, że kluczową rolę grają cechy ruchu sieciowegobrane pod uwagę w fazie treningowej i testowej. Rodzaje ataków "głośnych" takich jak DoS Hulk, DDos czy Hartbleed charakteryzują się dużą intensywnością i powtarzalnością. Zasada działania tych ataków generuje wyraźne różnice, odstające od normalnego ruchu. Wyraźnie zwiększa się liczba pakietów, częstotliwość połączeń czy rozmiar przesyłanych danych, co skutecznie jest wykrywane przez wszystkie analizowane algorytmy (tabela 9.2). Z drugiej strony ataki o krótkim czasie trwania, jak WEB Attack, Injection, czy Brute-force generują ruch bardzo zbliżonym do normalnego. Szczególnie widoczne jest to w modelach, które przez zasadę działania tworzą ogólny profil normalności, czyli Isolation Forest oraz Autoencoder. LOF wykazał się wyższą skutecznością w wykrywaniu tego typu ataków dzięki analizie lokalnej gęstości obserwacji w przestrzeni cech. Perspektywa LOF jest skupiona na lokalnych skupiskach przepływów, dzięki czemu lepiej widoczne są przepływy wakatów nawet gdy nie globalnie wpasowują się w normalny ruch wewnątrz sieci. Ta przewaga jest równocześnie przyczyną zwiększonej liczby fałszywych alarmów.

Obserwacje potwierdzają, że jakość detekcji anomalii w ruchu sieciowym nie zależy tylko od rodzaju wektora ataku, ale również od zaimplementowanego algorytmu. Wyniki te uzasadniają oczywistą ścieżkę dalszych badań, będącą łączeniem różnych podejść detekcji w celu wykorzystania ich komplementarnych właściwości. Uczenie nienadzorowane posiada potencjał do wykrywania ataków w ruchu sieciowym bez wcześniejszej znajomości sygnatur ataku. Badania potwierdzają potencjał, jednak również podkreślają ograniczenia. Narzędzie, w którym zaimplementowano modele uczenia nienadzorowanego do wykrywania ataku zero-day, ma jak najbardziej prawo bytu i potencjał do ochrony środowiska od nieznanym wcześniej zagrożeń z cyberprzestrzeni.

TABELA 9.1: Porównanie wyników wszystkich testowanych modeli

Miara	IF	LOF	Autoencoder
Accuracy	91,54%	79,92%	91,84%
Sensitivity	68,20%	88,91%	72,09%
Precision	86,66%	49,89%	84,81%
F-measure	76,33%	63,91%	77,93%

TABELA 9.2: Porównanie skuteczności wykrywania wybranych typów ataków

Typ ataku	IF [%]	LOF [%]	AE [%]
SSH-Patator	0,06	96,04	0,06
FTP-Patator	0,00	99,60	0,03
DoS Slowhttptest	15,23	90,43	1,741
DoS Hulk	86,47	96,28	89,79
DoS slowloris	10,59	60,55	0,02
DoS GoldenEye	58,16	83,57	54,58
Heartbleed	100,00	100,00	100,00
Web Attack XSS	2,91	2,76	0,61
Web Attack Brute Force	5,12	3,57	0,28
Web Attack SQL Injection	0,00	70,00	5,00
Infiltration	41,66	44,44	30,56
Bot	3,29	42,33	3,29
PortScan	3,85	72,96	14,20
DDoS	56,57	82,02	63,51

9.2 Kierunki dalszych prac

Przeprowadzone badania wykazały, że żaden z analizowanych algorytmów nie zapewnia jednocześnie wysokiej czułości i wysokiej precyzji dla wszystkich typów ataków. Wyniki te jednoznacznie wskazują, że modele te nie popełniają tych samych błędów. Ataki niewykryte przez Isolation Forest były często wykrywane przez LOF, natomiast przypadki fałszywych alarmów generowane przez LOF były w dużej mierze filtrowane przez pozostałe modele. Z punktu widzenia detekcji ataków typu zero-day, gdzie priorytetem jest wykrycie jak największej liczby rzeczywistych incydentów, takie zróżnicowanie zachowań modeli stanowi istotną przesłankę do ich agregacji. Naturalnym kierunkiem dalszych badań jest zatem eksperymentowanie z łączeniem wyników wielu detektorów anomalii. W ramach eksperymentu z posiadanymi wynikami modeli rozważono i przetestowano dwa podejścia do agregacji decyzji modeli.

1. Reguła OR – W pierwszym podejściu przepływ sieciowy był uznawany za anomalię, jeżeli co najmniej jeden z modeli zaklasyfikował go jako anomalię.

Zastosowana metoda skutkowała bardzo wysoką czułością detekcji, przekraczającą 89%, jednak jednocześnie znacznym spadkiem precyzji (48%) (tabela 9.3). Oznaczało to, że gotowy system generowałby ogromną liczbę FP, co w praktyce prowadziłoby do przeciążenia systemu, oraz istotnego obniżenia jego użyteczności operacyjnej. Powodem niepowodzenia okazał się LOF. Po zestawieniu wyników LOF z wynikami podejścia agregowanego zauważalne jest ich duże podobieństwo, co wskazuje, że model ten w praktyce zdominował regułę OR. Fałszywe detekcje generowane przez LOF były nakładane na poprawne decyzje pozostałych modeli, a dodatkowo wprowadzały nowe FP, które nie występowały w klasyfikacjach Isolation Forest ani Autoencodera (tabela 9.4).

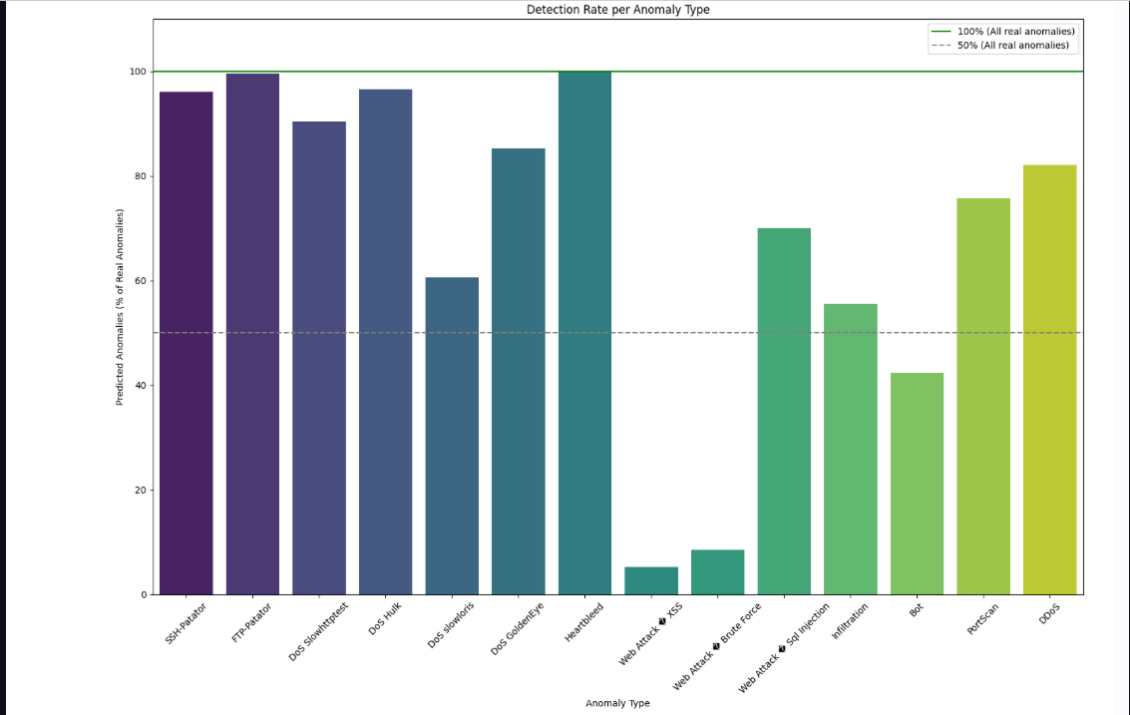
Mimo wysokiej skuteczności ataków, reguła OR, została uznana za nierealistyczną do implementacji i dalszego rozwoju.

TABELA 9.3: Wyniki miar jakości agregacji wyników reguły OR (zastosowano tą samą funkcję co do oceny pojedynczych modeli)

Miara	Wartość
Accuracy	78,59%
Sensitivity	89,15%
Precision	48,09%
F-measure	62,48%

TABELA 9.4: Skuteczność detekcji poszczególnych typów ataków przez agregowane wyniki regułą OR

Typ ataku	Poprawnie wykryte [%]
SSH-Patator	96,0355
FTP-Patator	99,6122
DoS Slowhttptest	90,4725
DoS Hulk	96,5315
DoS slowloris	60,5537
DoS GoldenEye	85,2835
Heartbleed	100,0000
Web Attack XSS	5,2147
Web Attack Brute Force	8,4793
Web Attack SQL Injection	70,0000
Infiltration	55,5556
Bot	42,3336
PortScan	75,6630
DDoS	82,0549



RYSUNEK 9.1: Wizualizacja wyników z tabeli 9.4

2. Selektywna agregacja – Drugie podejście polegało na bardziej konserwatywnym odwrotnym podejściu. Zasada ta działała na założeniu, że jeżeli wszystkie modele wykryły dany flow jako anomalie, to wykrycie jest prawidłowe. Przechodząc w tę skrajność, eksperyment miał na celu zademonstrowanie, że mimo że nie zawsze wszystkie nowe ataki zostaną wykryte to część z nich nadal może być wyłapana i to z dużą pewnością (tabela 9.5).

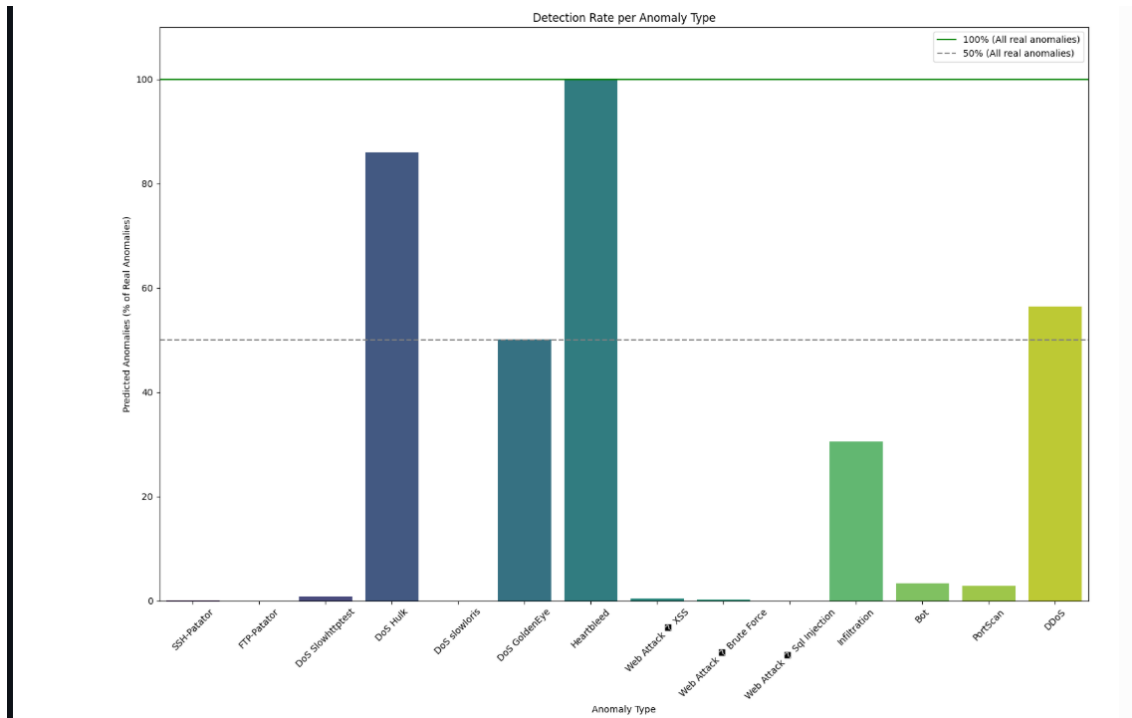
Takie podejście pozwoliło na zachowanie wysokiej precyzji detekcji na poziomie powyżej 91%, przy jednoczesnym utrzymaniu czułości na poziomie około 67%. W porównaniu do pojedynczych modeli uzyskano lepszy balans pomiędzy liczbą wykrytych ataków a liczbą fałszywych alarmów (tabela 9.6).

TABELA 9.5: Wyniki miar jakości wyników po Selektywnej Agregacji

Miara	Wartość
Accuracy	92,15%
Sensitivity	67,24%
Precision	91,16%
F-measure	77,39%

TABELA 9.6: Skuteczność detekcji poszczególnych typów ataków przez Selektywną Agregację

Typ ataku	Poprawnie wykryte [%]
SSH-Patator	0,0634
FTP-Patator	0,0000
DoS Slowhttptest	0,8227
DoS Hulk	85,9696
DoS slowloris	0,0000
DoS GoldenEye	50,1702
Heartbleed	100,0000
Web Attack XSS	0,4601
Web Attack Brute Force	0,2102
Web Attack SQL Injection	0,0000
Infiltration	30,5556
Bot	3,2909
PortScan	2,8081
DDoS	56,4974



RYSUNEK 9.2: Wizualizacja wyników z tabeli 9.6

W dalszych pracach możliwe jest rozwinięcie drugiego podejścia poprzez wprowadzenie dynamicznego ważenia decyzji poszczególnych modeli lub adaptacyjne dostosowywanie progów detekcji do zmieniającego się charakteru ruchu sieciowego. Dodatkowym kierunkiem rozwoju jest integracja informacji kontekstowych, takich jak czas trwania sesji, częstotliwość połączeń czy zachowanie użytkownika, które mogą dostarczyć dodatkowych sygnałów wspomagających ocenę anomalii.

Uzyskane wyniki wskazują, że systemy oparte na agregacji wielu detektorów anomalii stanowią obiecujący kierunek rozwoju metod wykrywania nieznanych zagrożeń w ruchu sieciowym. Połączenie modeli o różnych charakterystykach detekcji pozwala na częściowe uzupełnianie ich indywidualnych słabości oraz lepsze wykorzystanie ich mocnych stron. W ujęciu teoretycznym dalszy rozwój narzędzi agregujących wiele modeli wykrywania anomalii, opartych na uczeniu nienadzorowanym, mógłby prowadzić do systemów coraz lepiej dopasowanych do specyfiki chronionej sieci. Dynamiczne dostosowywanie się modeli do normalnego profilu ruchu umożliwiłoby coraz dokładniejsze rozróżnianie normalnych flow od antynomijnych, co w konsekwencji mogłoby pozwolić na skuteczniejsze generowanie alertów o zagrożeniach i atakach zachodzących w sieci w czasie rzeczywistym.

Spis rysunków

3.1	Ilustracja działania algorytmu. Na czerwono zaznaczone zostały anomalie a na niebiesko normalne dane. a - cięcia, b- struktura drzewa [8]	8
3.2	Ilustracja zasady działania KNN [11]	9
3.3	Ilustracja architektury Autoencodera przedstawiona na podstawie wejściowej danej rysunku liczby „4”i jej odtworzenia. W kontekście wykrywania anomalii Autoenkoder uczy się odtwarzać poprawne przykłady danych w tym przypadku cyfrę „4”. Jeżeli rekonstrukcja istotnie różni się od obrazu wejściowego, może to wskazywać, że na wejściu pojawiła się anomalia, czyli obiekt różniący się od danych, na których model był trenowany [13]	10
4.1	Wycinek kodu ilustrujący implementację modelu IF, mający na celu przedstawienie składni	14
5.1	Wycinek kodu ilustrujący implementacje LOF, mający na celu przedstawienie składni	18
6.1	Wycinek kodu ilustrujący implementacje Autoencodera, mający na celu przedstawienie zasady implementacji w badaniu	21
6.2	Wycinek kodu ilustrujący implementacje fazę treningową Autoencodera, mający na celu przedstawienie zasady implementacji w badaniu	22
7.1	Statystyki pochodzące z badania ważności cech ilustrowane na wykresach Źródło: K. Kostas, Anomaly Detection in Networks Using Machine Learning, 2021 [6]	28
8.1	Wykres ilustrujący wyniki modelu Isolation Forest w ramach Pipeline	35
8.2	Wykres ilustrujący wyniki modelu Local Outlier Factor w ramach Pipeline	36
8.3	Wykres ilustrujący wyniki modelu Autoencoder w ramach Pipeline	38
9.1	Wizualizacja wyników z tabeli 9.4	41
9.2	Wizualizacja wyników z tabeli 9.6	43

Spis tabel

7.1	Dystrybucja ilościowa przepływów oznaczonych i przypisanych do poszczególnych ataków	27
7.2	Reprezentacja wyników badania istotności cech dla zbioru CIC-IDS2017. Źródło: K. Kostas, Anomaly Detection in Networks Using Machine Learning, 2021 [6]	28
7.3	Intuicyjne wyjaśnienie cech branych pod uwagę w trenowaniu i testowaniu modeli [6]	29
8.1	Wyniki miar jakości modelu Isolation Forest	34
8.2	Skuteczność detekcji poszczególnych typów ataków przez model Isolation Forest	35
8.3	Wyniki miar jakości modelu Local Outlier Factor	36
8.4	Skuteczność detekcji poszczególnych typów ataków przez model Local Outlier Factor	37
8.5	Wyniki miar jakości modelu Autoencoder	38
8.6	Skuteczność detekcji poszczególnych typów ataków przez model Autoencoder	38
9.1	Porównanie wyników wszystkich testowanych modeli	40
9.2	Porównanie skuteczności wykrywania wybranych typów ataków	40
9.3	Wyniki miar jakości agregacji wyników reguły OR (zastosowano tę samą funkcję co do oceny pojedynczych modeli)	41
9.4	Skuteczność detekcji poszczególnych typów ataków przez agregowane wyniki regułą OR	41
9.5	Wyniki miar jakości wyników po Selekttywnej Agregacji	42
9.6	Skuteczność detekcji poszczególnych typów ataków przez Selektywną Agregację	42

Literatura

- [1] Jerzy Zawisza. „Cyberprzestrzeń jako zagrożenie bezpieczeństwa państwa”. *Journal of Modern Science*, 27(4):403–416, 2015.
- [2] U.S. Department of Homeland Security (DHS) Cybersecurity and Infrastructure Security Agency (CISA). „CVE-2021-26855”. <https://www.cve.org/CVERecord?id=CVE-2021-26855>, 2021. [Data uzyskania dostępu 29-Października-2025].
- [3] U.S. Department of Homeland Security (DHS) Cybersecurity and Infrastructure Security Agency (CISA). „CVE-2021-26857”. <https://www.cve.org/CVERecord?id=CVE-2021-26857>, 2021. [Data uzyskania dostępu 29-Października-2025].
- [4] U.S. Department of Homeland Security (DHS) Cybersecurity and Infrastructure Security Agency (CISA). „CVE-2021-27065”. <https://www.cve.org/CVERecord?id=CVE-2021-27065>, 2021. [Data uzyskania dostępu 29-Października-2025].
- [5] Canadian Centre for Cyber Security. „Alert - Active Exploitation of Microsoft Exchange Vulnerabilities - update 4”. <https://www.cyber.gc.ca/en/alerts-advisories/active-exploitation-microsoft-exchange-vulnerabilities>, 2021. [Data uzyskania dostępu 29-Października-2025].
- [6] Kahraman Kostas. „anomaly detection in networks using machine learning”. Master’s thesis, School of Computer Science and Electronic Engineering, University of Essex, 2018.
- [7] SAP SE. „What is Machine Learning?”. <https://www.sap.com/poland/products/artificial-intelligence/what-is-machine-learning.html>, 2025. [Data uzyskania dostępu 26-Październik-2025].
- [8] Abdelkader Dairi, Fouzi Harrou, and Ying Sun. „efficient driver drunk detection by sensors: A manifold learning-based anomaly detector”. *IEEE Access*, PP:1–1, 11 2022.
- [9] Fei Tony Liu, Kai Ting, and Zhi-Hua Zhou. Isolation forests. In „*Efficient Driver Drunk Detection by Sensors: A Manifold Learning-Based Anomaly Detecto*”, pages 413 – 422, 01 2009.
- [10] Abubakar Fadul. „*Anomaly Detection based on Isolation Forest and Local Outlier Factor*”. PhD thesis, African Institute for Mathematical Sciences, 05 2023.
- [11] GeeksforGeeks. „How to reduce KNN computation time using KD-Tree or Ball Tree?”. <https://www.geeksforgeeks.org/machine-learning/how-to-reduce-knn-computation-time-using-kd-tree-or-ball-tree/>, 2025. [Data uzyskania dostępu 18-Stycznia-2026].
- [12] Dagang Wei. „Demystifying Neural Networks: Anomaly Detection with AutoEncoder”. <https://medium.com/@weidagang/demystifying-anomaly-detection-with-autoencoder-neural-networks-1e235840d879>, 2024. [Data uzyskania dostępu 29-Października-2025].
- [13] Robin Teuwens. „Anomaly Detection with Auto-Encoders”. <https://www.kaggle.com/code/robinteuwens/anomaly-detection-with-auto-encoders1>, 2020. [Data uzyskania dostępu 29-Października-2025].

- [14] Scikit-learn Developers. „Scikit-learn Documentation „sklearn.ensemble.IsolationForest””. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>, 2025. [Data uzyskania dostępu 04-Stycznia-2026].
- [15] R.Bhasker. „Mastering Isolation Forest: A Deep Dive into Anomaly Detection”. <https://medium.com/%40Rahul.bhasker/mastering-isolation-forest-a-deep-dive-into-anomaly-detection-db973d9290f1>, 2025. [Data uzyskania dostępu 22-Stycznia-2026].
- [16] M. Brenndoerfer. „Isolation Forest: Complete Guide to Unsupervised Anomaly Detection with Random Trees & Path Length Analysis”. https://mbrenndoerfer.com/writing/isolation-forest-anomaly-detection-unsupervised-learning-random-trees-path-length-protect%40normalcr%2Frelax-mathematical-foundations-python-scikit-learn-guide?utm_source, 2025. [Data uzyskania dostępu 22-Stycznia-2026].
- [17] Scikit-learn Developers. „Scikit-learn Documentation „sklearn.neighbors.LocalOutlierFactor””. <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.LocalOutlierFactor.html#sklearn.neighbors.LocalOutlierFactor>, 2025. [Data uzyskania dostępu 05-Stycznia-2026].
- [18] Belwin J. Brearley, K. Regin Bose, K. Senthil, and G. Ayyappan. „enhancing multi-class knn in ball-trees using adaptive pruning”. *International Journal of Intelligent Systems and Applications in Engineering*, 12(13s), 2024.
- [19] Scikit-learn Developers. „Scikit-learn Documentation „Documentation for an old release of Scikit-learn (version 0.24)””. <https://scikit-learn.org/0.24/modules/neighbors.html>, 2020. [Data uzyskania dostępu 18-Stycznia-2026].
- [20] Abdulaziz Alqurashi et al. „a review of local outlier factor algorithms for outlier detection in big data streams”. *Big Data and Cognitive Computing*, 5.1.1, 2021.
- [21] GeeksforGeeks. „Implementing an Autoencoder in PyTorch”. <https://www.geeksforgeeks.org/deep-learning/implementing-an-autoencoder-in-pytorch/>, 2025. [Data uzyskania dostępu 18-Stycznia-2026].
- [22] D.Bergmann, C.Stryker. „What is an autoencoder?”. <https://www.ibm.com/think/topics/autoencoder>, 2025. [Data uzyskania dostępu 18-Stycznia-2026].
- [23] Canadian Institute for Cybersecurity. „Intrusion detection evaluation dataset (CIC-IDS2017)”. <https://www.unb.ca/cic/datasets/ids-2017.html>, 2017. [Data uzyskania dostępu 22-Stycznia-2026].
- [24] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. „toward generating a new intrusion detection dataset and intrusion traffic characterization”. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116, Portugal, January 2018. SCITEPRESS.
- [25] Ranjit Panigrahi and Samarjeet Borah. „a detailed analysis of cicids2017 dataset for designing intrusion detection systems”. *International Journal of Engineering & Technology*, 7:479–482, 01 2018.
- [26] Lanjelot. „patator: Multi-purpose brute-forcer”. <https://github.com/lanjelot/patator>, 2024. Data uzyskania dostępu 22-Stycznia-2026.
- [27] A.Michalczyk. „ataki slow http dos (cz. 1.) – slowloris”. <https://sekurak.pl/ataki-slow-http-dos-cz-1-slowloris/>, 2014. Data uzyskania dostępu 22-Stycznia-2026.

- [28] A.Michalczyk. „ataki slow http dos (cz. 2.) — powolne dosyłanie ciała żądania w ataku slow http body przy użyciu slowhttpptest”.
<https://sekurak.pl/ataki-slow-http-dos-cz-2-slow-http-body-slowhttpptest/>, 2014. Data uzyskania dostępu 22-Stycznia-2026.
- [29] Radwarek. „ddospedia - hulk”.
<https://www.radware.com/security/ddos-knowledge-center/ddospedia/hulk/>, 2026. Data uzyskania dostępu 22-Stycznia-2026.
- [30] Vasileiadis A. „guide to ddos attacks with goldeneye”.
<https://medium.com/@redfanatic7/guide-to-ddos-attacks-with-goldeneye-3c2444011f82>, 2024. Data uzyskania dostępu 22-Stycznia-2026.
- [31] CloudFlare. „what is the low orbit ion cannon (loic)?”. <https://www.cloudflare.com/en-gb/learning/ddos/ddos-attack-tools/low-orbit-ion-cannon-loic/>, 2024. Data uzyskania dostępu 22-Stycznia-2026.
- [32] OWASP KirstenS. „cross site scripting (xss)”. <https://owasp.org/www-community/attacks/xss/>, 2025. Data uzyskania dostępu 23-Stycznia-2026.
- [33] Inc. Blackduck. „the heartbleed bug”. <https://www.heartbleed.com/>, 2025. Data uzyskania dostępu 23-Stycznia-2026.
- [34] Fortinet. „what is a port scan? how to prevent port scan attacks?”.
<https://www.fortinet.com/resources/cyberglossary/what-is-port-scan>, 2026. Data uzyskania dostępu 23-Stycznia-2026.
- [35] Google for Developers. „classification: Accuracy, recall, precision, and related metrics”.
<https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall>, 2026. Data uzyskania dostępu 23-Stycznia-2026.



© 2026 Autor: Michał Kowalski

Instytut Informatyki, Wydział Informatyki i Telekomunikacji
Politechnika Poznańska

Skład przy użyciu systemu $\text{\LaTeX}$ na platformie Overleaf.